

Othello AI

TIPE | Louis H. | 00000

Introduction



Deep Blue contre Garry Kasparov, 1997, kasparov.com (modifiée)



AlphaGo contre Lee Sedol, 2016, Lee Jin-Man (modifiée)

Titre | Problématique

Maîtriser informatiquement le jeu de l'Othello sans connaissances humaines.

Comment concevoir un programme informatique capable d’apprendre à jouer au jeu de l’Othello en utilisant des réseaux de neurones artificiels entraînés sans données humaines, dans le but d’optimiser et de surpasser les méthodes habituellement utilisées pour résoudre ce jeu ?

Sommaire

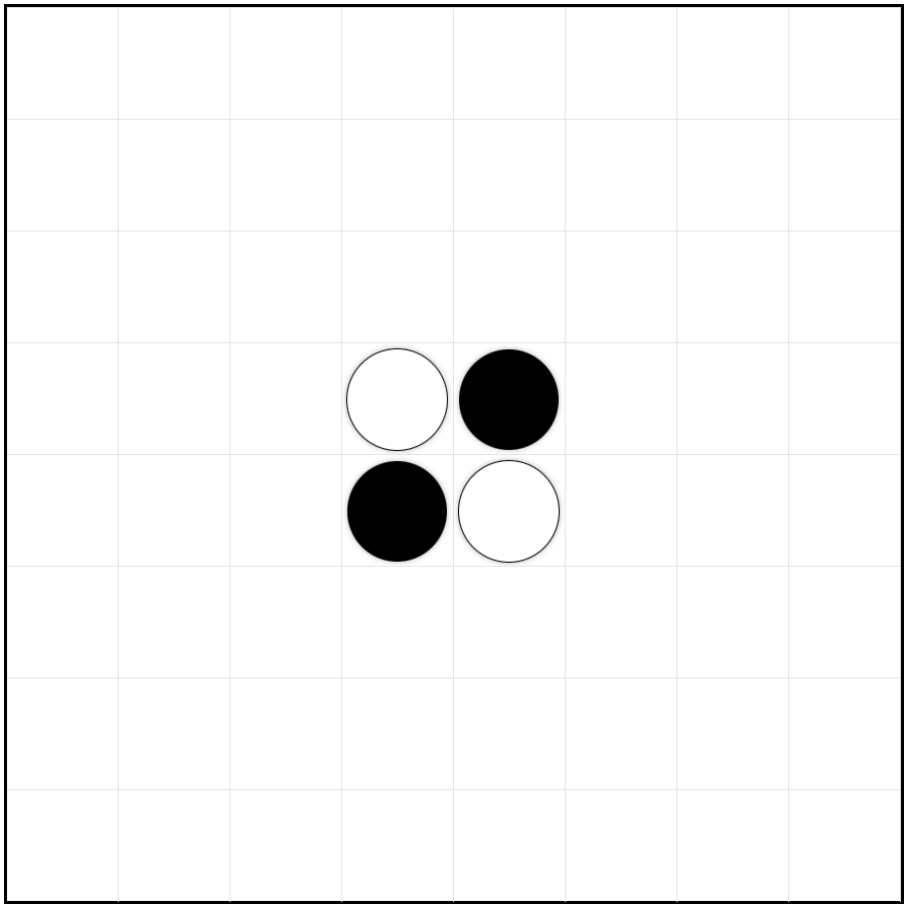
- L’Othello
- Première approche
- Réseaux de neurones
- Approche améliorée
- Résultats
- Annexe

Jeu de l'Othello

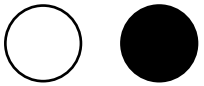
Règles du jeu | Implémentation

Jeu de l'Othello

Principe et règles du jeu

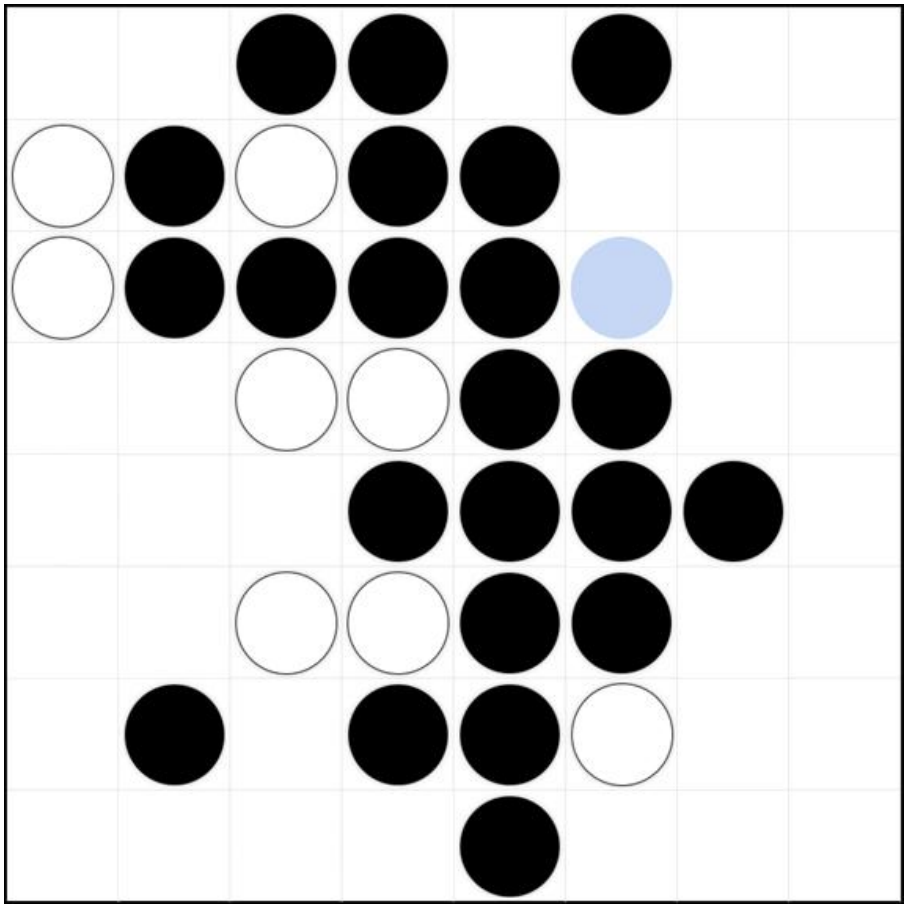


Othello

- 2 joueurs 
- Capture en ligne, colonne et diagonales

Jeu de l'Othello

Principe et règles du jeu

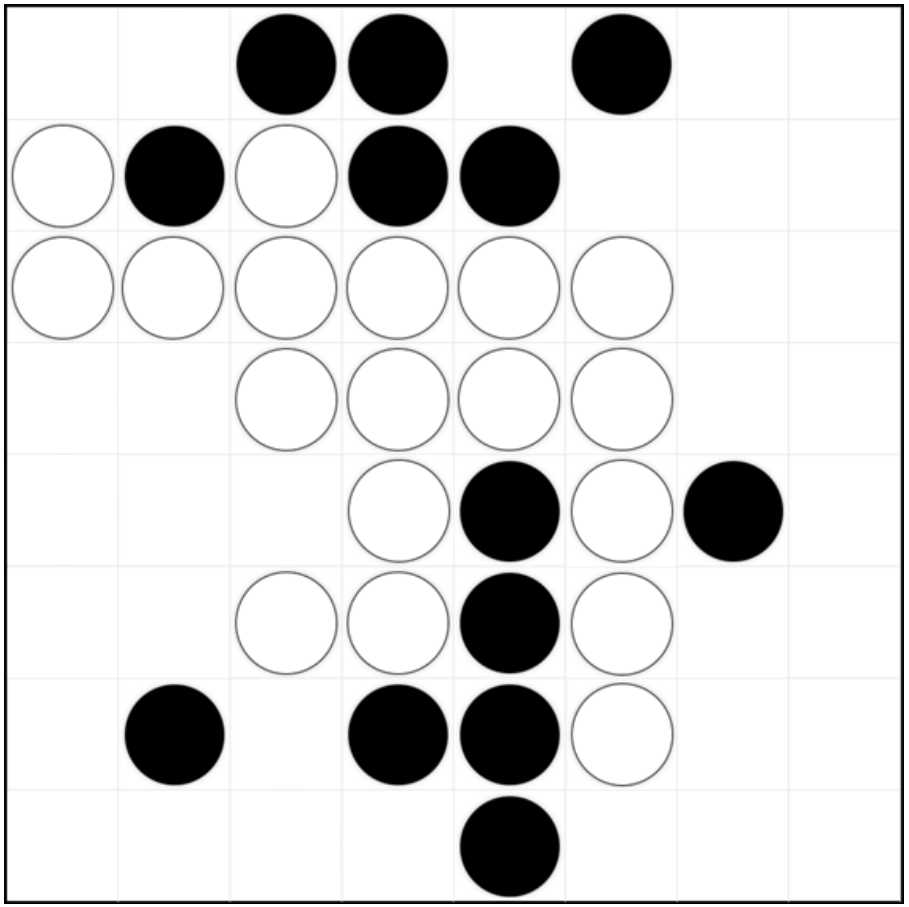


Exemple

- 26 coups joués
- Score noir : 22
- Score blanc : 8

Jeu de l'Othello

Principe et règles du jeu

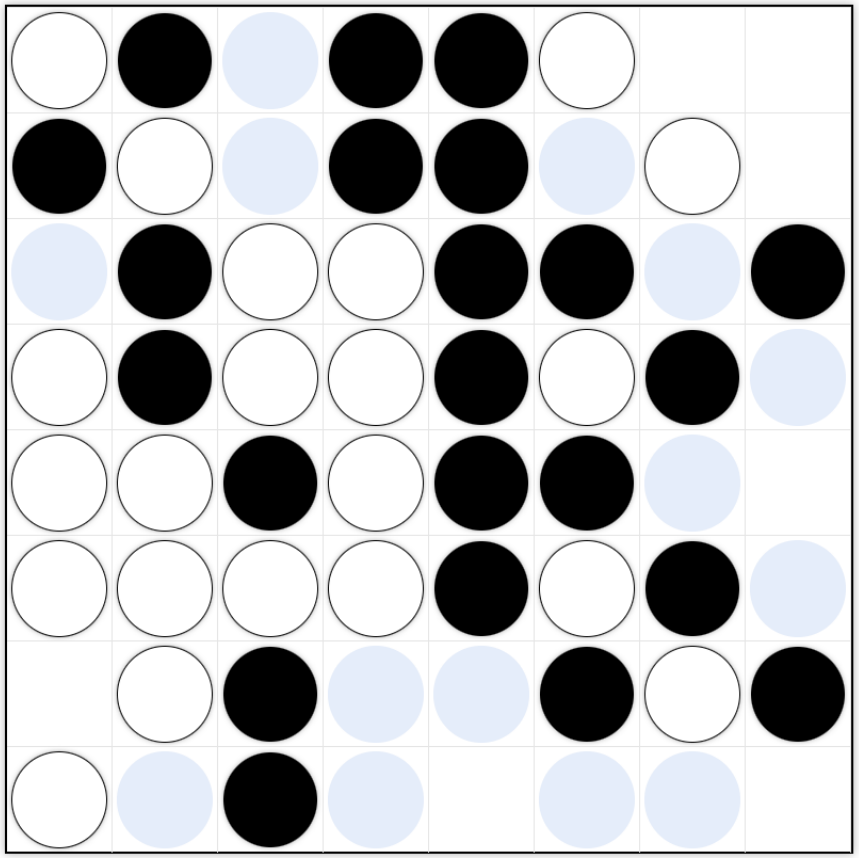


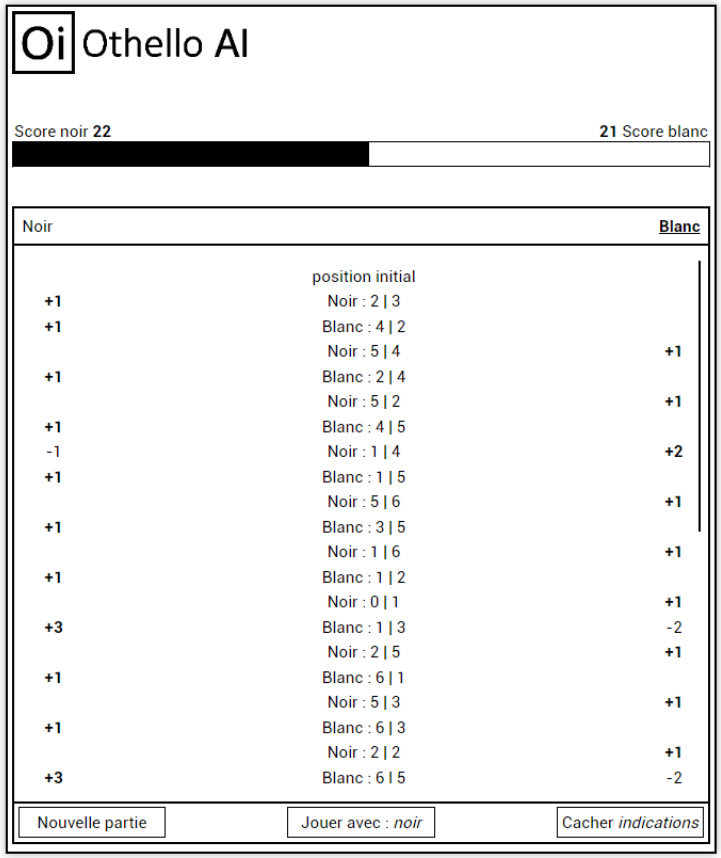
Exemple

- 27 coups joués
- Score noir : 13
- Score blanc : 18

Joueur à l'Othello

Implémentation en Python, HTML, CSS et JavaScript



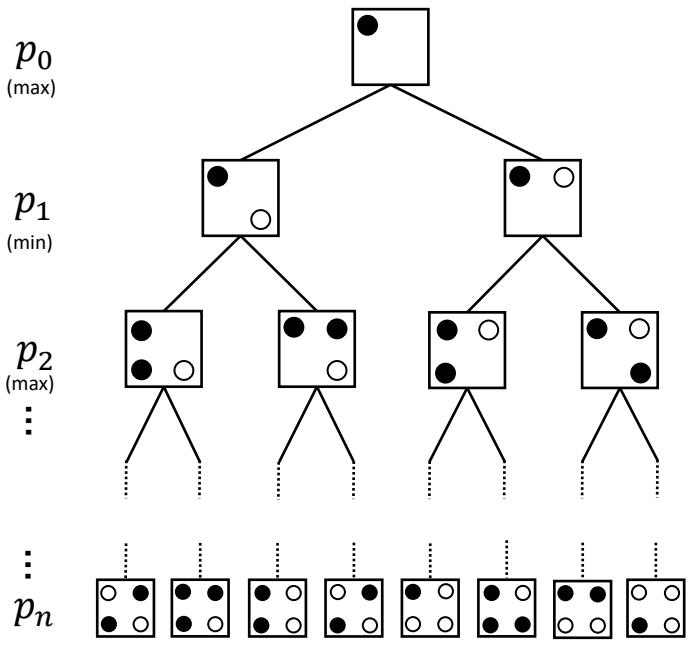


Première approche

Algorithme MinMax | Heuristique

Première approche

Arbre de recherche

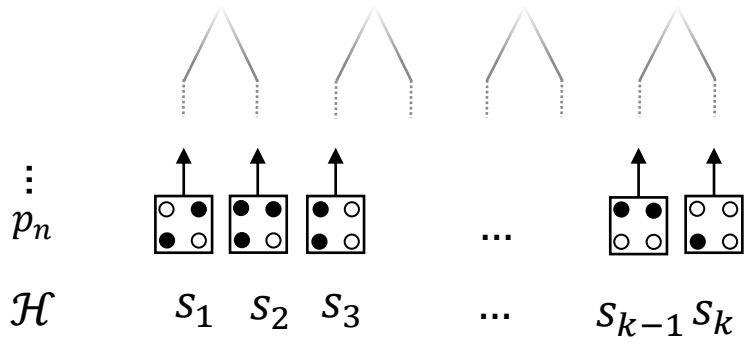


Heuristique

$\mathcal{H} : B \rightarrow \mathbb{R}$ avec B l'ensemble des plateaux d'Othello possible

$$\mathcal{H}\left(\begin{array}{|cc|}\hline \bullet & \circ \\ \bullet & \\ \hline \end{array}_{\,i}\right) = s_i$$

Plus s est grande plus il est possible que le programme gagne sur le plateau



Première approche

Résultats

Méthodes utilisées

- Programmé en C++
- $\mathcal{H}$: Nombre de pièce de sa propre couleur
- Recherche en profondeur
- Profondeur de recherche : 8
- Méthode des *bitboard*
- Table de hachage

Résultats

- 2-3 millions coups/s
- Temps de jeu moyens 1,3 secondes

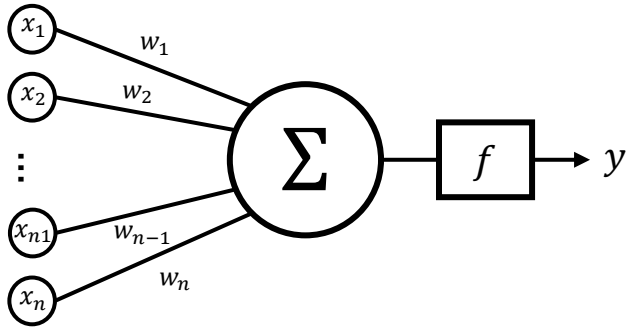
Problématiques

- Grand nombre de coup à chaque étape
- Heuristique imparfaite
- Recherche peu profonde

Réseaux de neurones

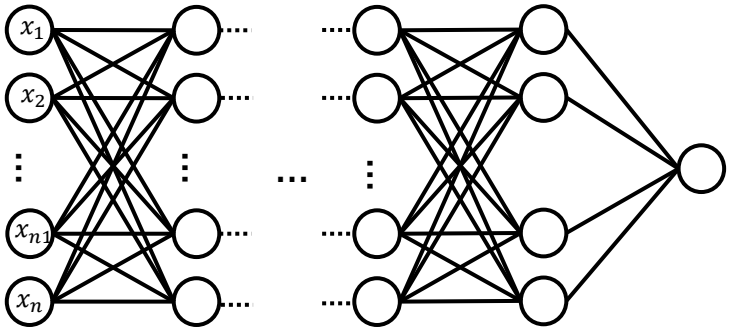
Neurone | Réseaux | Convolution

Réseau de neurones artificiels

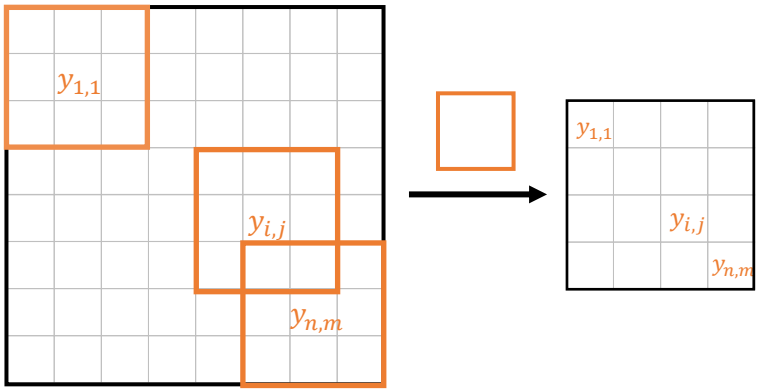


$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Réseaux de neurones :



Convolution :



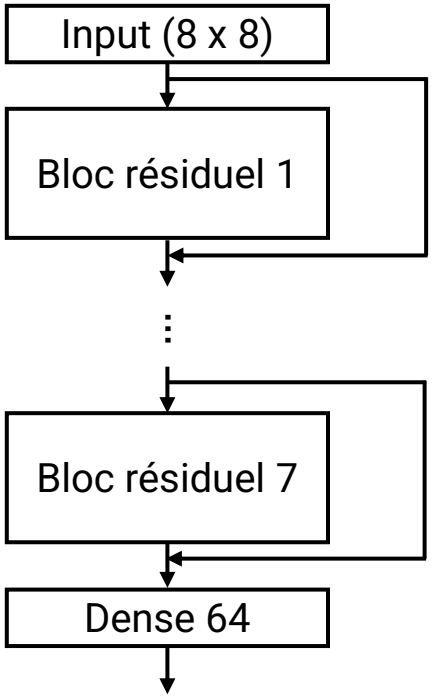
Approche améliorée

Modèles | Implémentation | Apprentissage

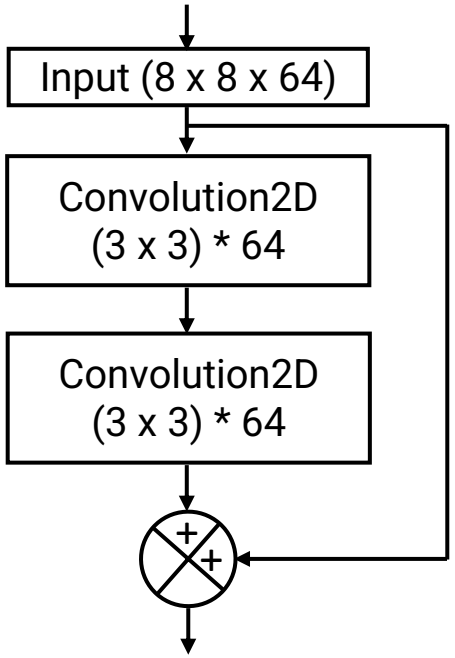
Approche améliorée

Première version

Réseau politique (policy network)



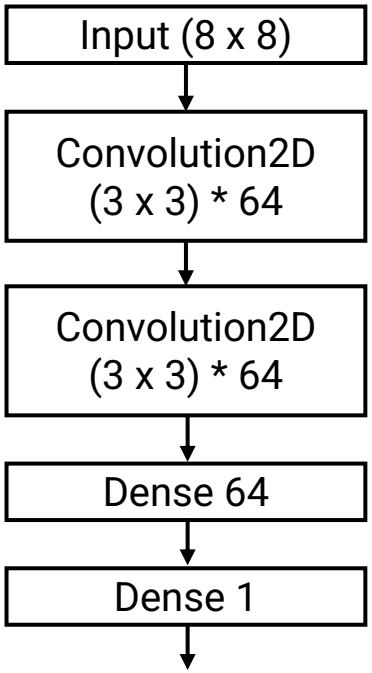
Bloc résiduel



Approche améliorée

Première version

Réseau de valeur (value network)



Fonctionnement

Value network

Joue le rôle de l'heuristique. Renvoie un flottant comprit entre 0 et 1.

Policy network

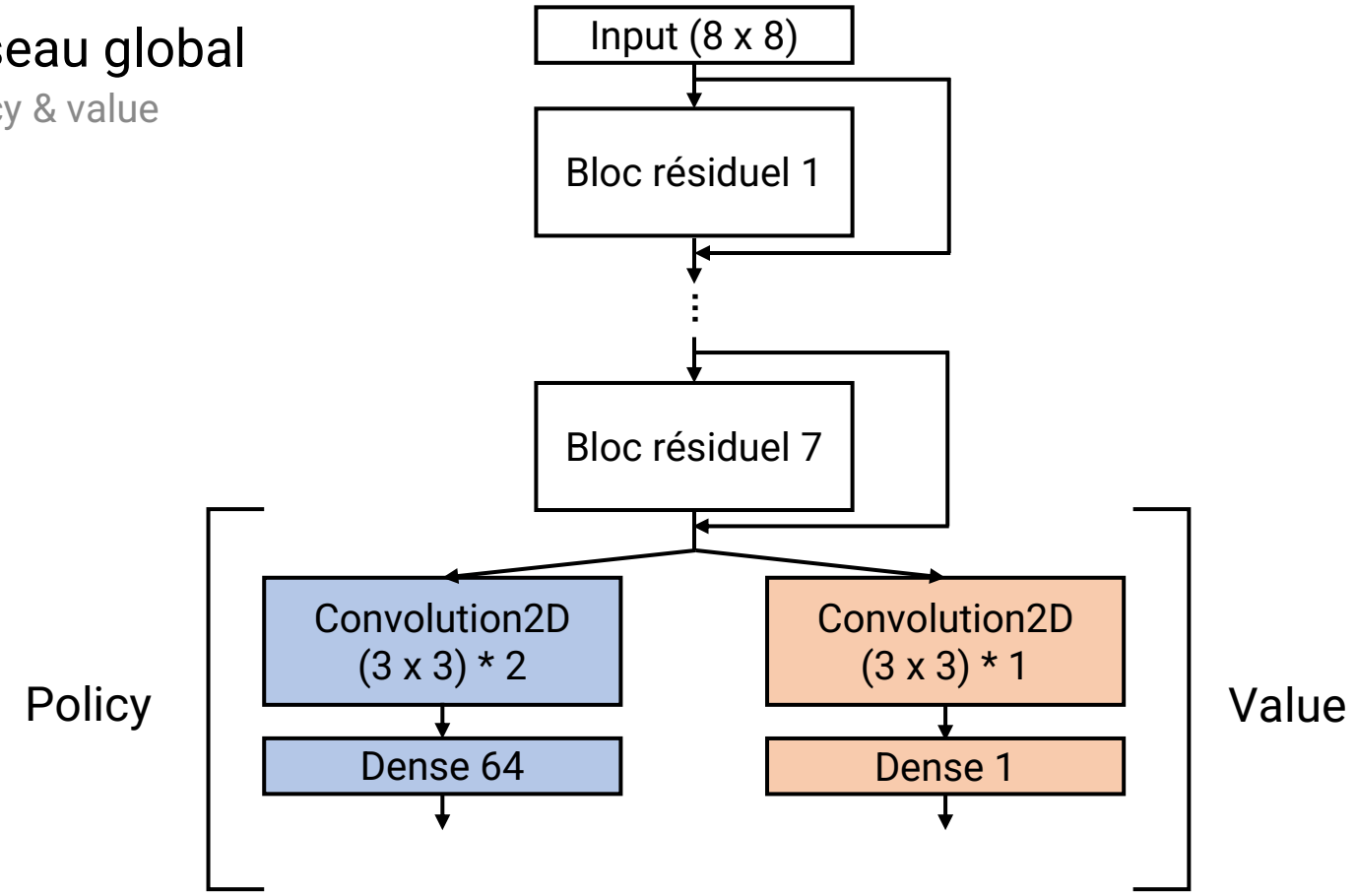
Indique quels sont les coups les plus pertinents à explorer.
Renvoie 64 flottants comprit entre 0 et 1 indiquant la pertinence.

Approche améliorée

Seconde version

Réseau global

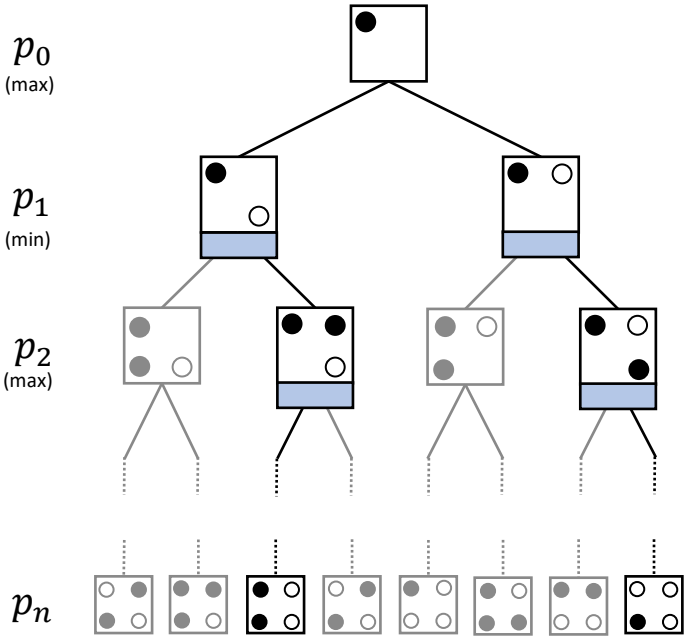
Policy & value



Approche améliorée

Implémentation

Policy network

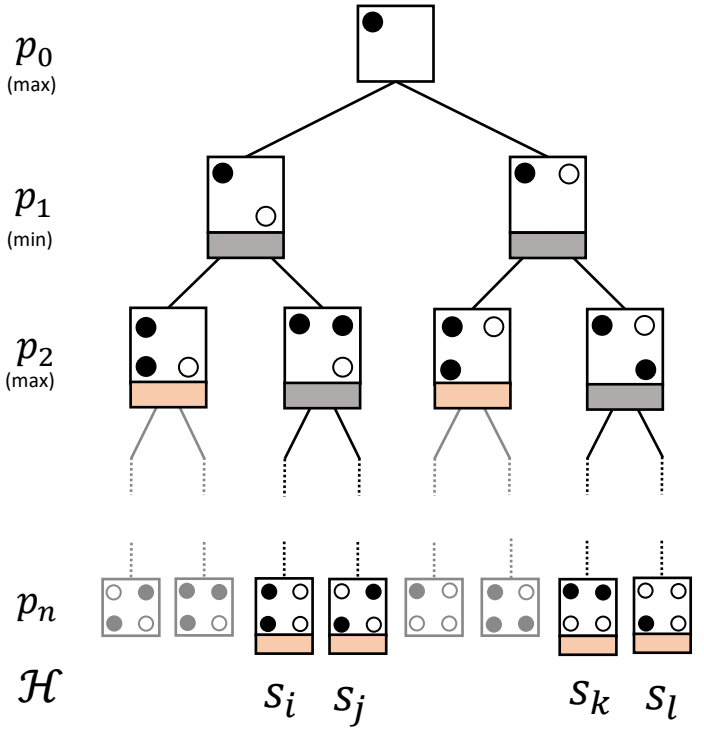


Utilisation Policy Network

Utilisation Value Network

Utilisation Value Network *inefficace*

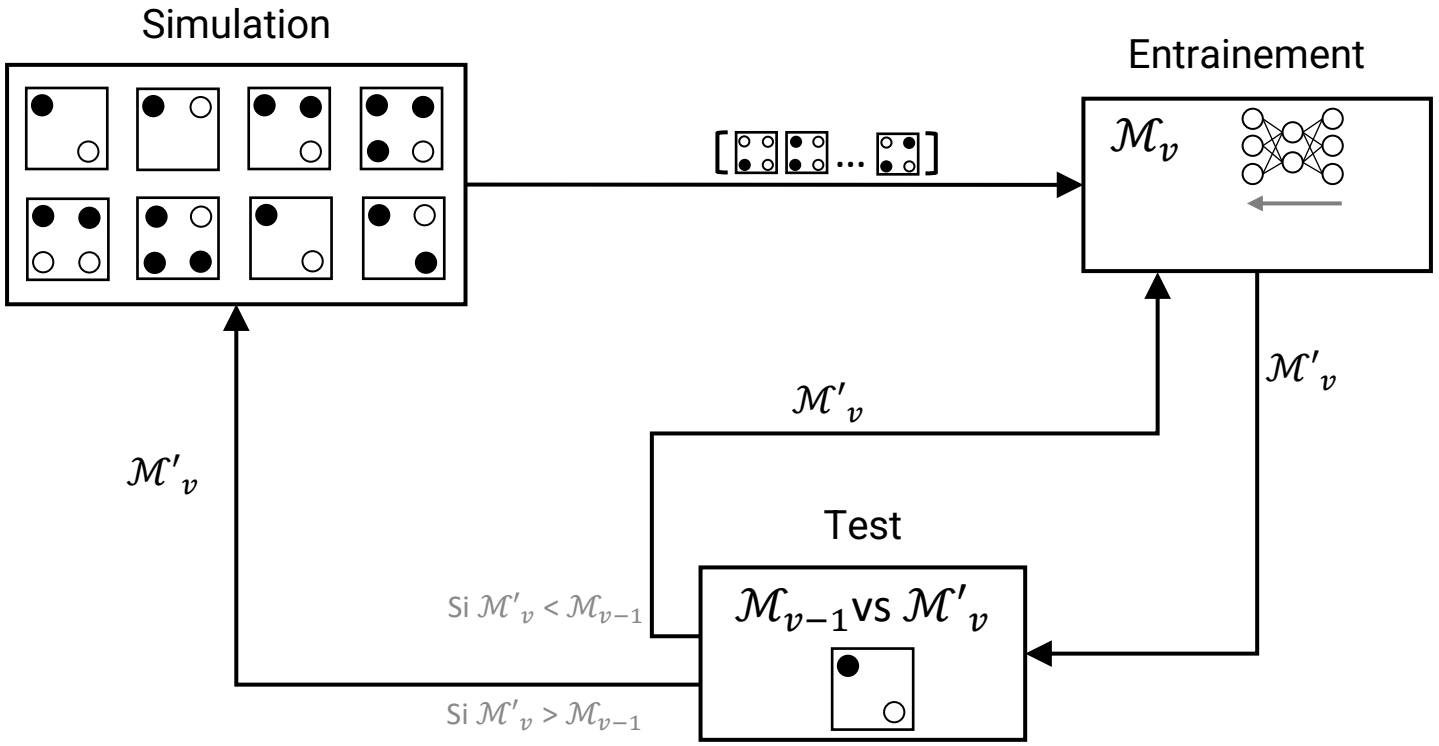
Value network



Approche améliorée

Entraînement

Apprentissage par renforcement :



Résultats

Apprentissage | Performances | Méthodes

Résultats

Configuration des apprentissages

Première version :

Policy network et Value network

Nombre de partie joués	Configuration
0 → 2500	p_{10} et 75% d'aléatoire
2500 → 10 000	p_{16} et 75% d'aléatoire
10 000 → 11 500	p_{16} et 50% d'aléatoire
11 500 → 18 000	p_{16} et **70%** d'aléatoire
18 000→ 32 000	p_{16} et 50% d'aléatoire

Réalisé pendant 15 jours

Victoire contre le modèle en C++

- **Version 1 :** Après 21 000 parties
- **Version 2 :** Après 8000 parties

Seconde version :

Réseau global

- Entrainée sur les parties de la première version
- ~5 350 000 Plateaux d'Othello
- ~100 000 Parties d'Othello

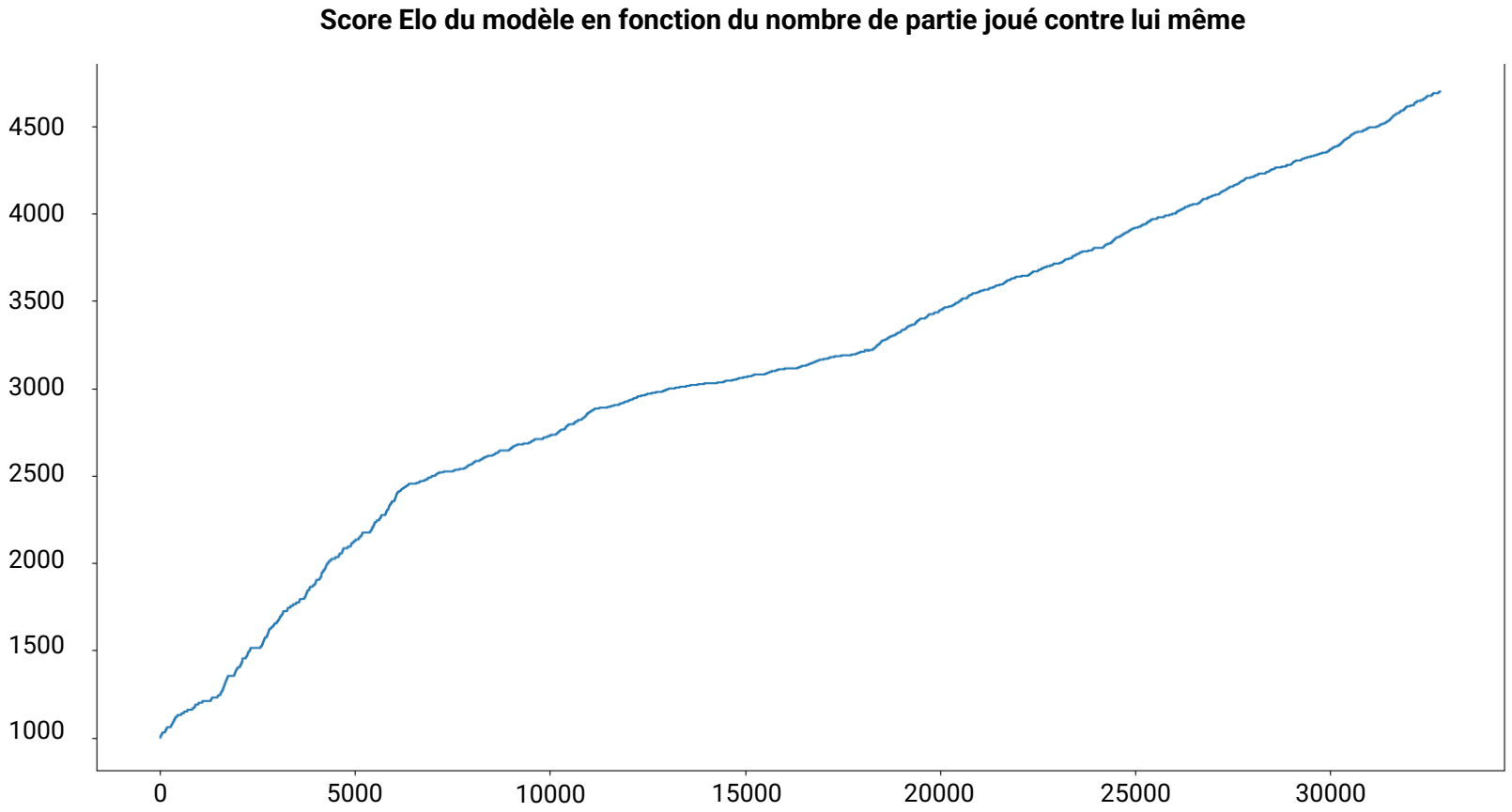
Réalisé en 1h

Nombre de partie joués	Configuration
0 → 4000	p_{10} et 75% d'aléatoire
4000 → 8000	p_{10} et 50% d'aléatoire
8000 → 20 000	p_{12} et 33% d'aléatoire
20 000 → 26 000	p_{16} et 20% d'aléatoire

Réalisé pendant 10 jours

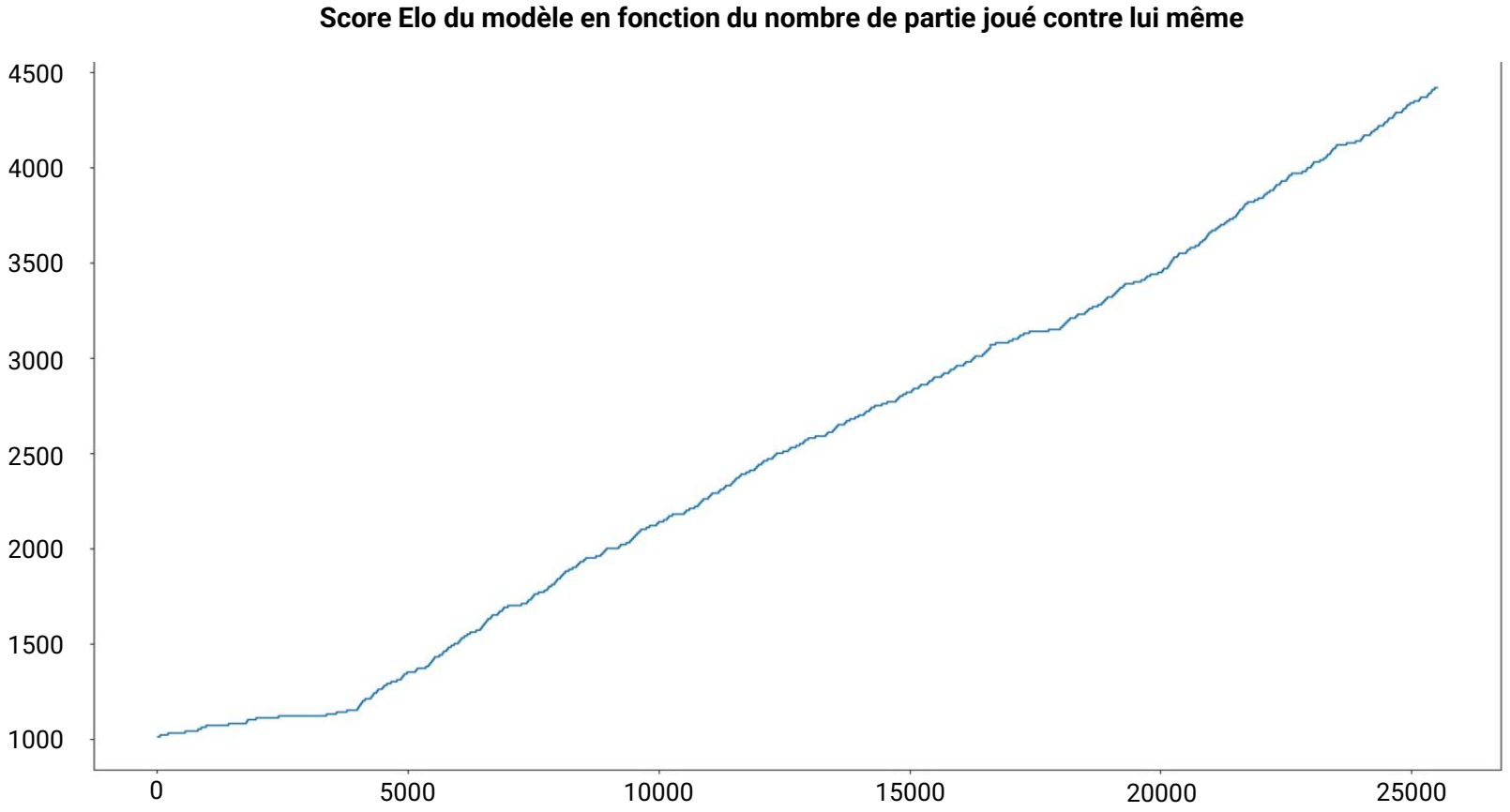
Résultats

Score Elo | Modèle 1



Résultats

Score Elo | Modèle 2



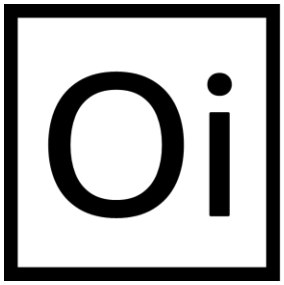
Conclusion

Réponse à la problématique

Conclusion

Maîtriser informatiquement le jeu de l'Othello sans connaissances humaines.

Comment concevoir un programme informatique capable d’apprendre à jouer au jeu de l’Othello en utilisant de l’intelligence artificielle et sans aucune donné humaines, dans le but d’optimiser et de surpasser les méthodes habituellement utilisées pour résoudre ce jeu ?



Othello AI

TIPE | Louis H. | 00000

Annexe

Code source | détails apprentissage

Code source

	Diapo	Listing
• cpp_engine		
• __init__.py	**29**	**15**
• OthelloEngine.cpp	**34**	**18**
• static		
• board.css	**45**	**25**
• game.js	**49**	**27**
• panel.css	**57**	**32**
• templates		
• board.html	**65**	**36**
• index.html	**69**	**37**
• tensorflow_engine		
• __init__.py	**73**	**39**
• /		
• evaluate_model.py	**82**	**45**
• live_graph.py	**87**	**48**
• main.py	**91**	**50**
• othello.py	**98**	**54**
• train_model.py	**107**	**58**

Détails modèles et apprentissage

• Modèle 1	**118**	**65**
• Modèle 2	**124**	**73**

CPP_engine/__init__.py

Implémentation première approche

```
from copy import deepcopy
import ctypes
from multiprocessing import Manager
from multiprocessing import Process

import numpy as np
from numpy.ctypeslib import ndpointer

COLOR_WHITE: int = 1
COLOR_BLACK: int = 2


class CppOthelloEngine:
    def __init__(self, max_depth: int, n_thread: int = 1, logs: bool = True) -> None:
        self.n_thread: int = n_thread
        self.logs: bool = logs
        self.max_depth: int = max_depth

    @staticmethod
    def board_to_bitboard(board: list[list[int]]) -> tuple:

        white_board: int = 0
        black_board: int = 0

        for y, line in enumerate(board):
            for x, val in enumerate(line):

                if val == COLOR_WHITE:
                    white_board += (2 ** ((7 - y) * 8 + (7 - x)))

                elif val == COLOR_BLACK:
                    black_board += (2 ** ((7 - y) * 8 + (7 - x)))

        return white_board, black_board

    def thread_search(self, board: np.array, thread_id: int, available: list[int], reward: list[int], engine_white:
bool):
        white_board, black_board = self.board_to_bitboard(board)

        stats = ctypes.c_int(0)
```

```
        r: int = get_move_reward(ctypes.c_ulonglong(white_board),
                                 ctypes.c_ulonglong(black_board),
                                 ctypes.c_bool(not engine_white),   # Black becomes white
                                 ctypes.c_int32(self.max_depth),
                                 stats)

        reward[thread_id] = r

        # end
        available[0] += 1

    def __call__(self, board: list[list[int]], engine_value: int, **kwargs) -> tuple:

        white_board, black_board = self.board_to_bitboard(board)

        engine_white = True if engine_value == COLOR_WHITE else False

        if self.n_thread == 1:
            stats = ctypes.c_int(0)
            r: int = get_best_move(ctypes.c_ulonglong(white_board),
                                   ctypes.c_ulonglong(black_board),
                                   ctypes.c_bool(engine_white),
                                   ctypes.c_int32(self.max_depth),
                                   stats)

            if self.logs:
                print(f"Move calculated: {stats.value:,}")

            # x, y
            return r % 8, r // 8

        else:

            manager = Manager()
            reward = manager.dict()
            available_thread = manager.list([self.n_thread, ])

            list_process = []

            # Simulate every possible move
```

```
        b = np.array(board, dtype=np.int32)

        for i, v in enumerate(legal_moves_array(b, engine_value).flatten()):
            if v == 1:
                new_board = place_piece(b, i, engine_value)

                while available_thread[0] <= 0:
                    pass

                available_thread[0] -= 1

                list_process.append(
                    Process(target=self.thread_search,
                            args=(new_board, i, available_thread, reward, engine_white,)
                            )
                )
                list_process[-1].start()

        for p in list_process:
            p.join()

        # cherche best move
        min_reward = 100
        min_pos = -1

        for pos in reward:
            if reward[pos] < min_reward:
                min_reward = reward[pos]
                min_pos = pos

        return min_pos % 8, min_pos // 8


try:
    lib = ctypes.cdll.LoadLibrary(__file__.replace("__init__.py", "OthelloEngine.so"))

    get_best_move = lib.cpp_get_best_move
    get_best_move.restype = ctypes.c_int32
    get_best_move.argtypes = [ctypes.c_ulonglong,   # white board
                              ctypes.c_ulonglong,   # black board
```

```
                                ctypes.c_bool,   # white engine
                                ctypes.c_int32,   # max depth
                                ctypes.POINTER(ctypes.c_int32)   # stats
                                ]

    get_move_reward = lib.cpp_get_move_reward
    get_move_reward.restype = ctypes.c_int32
    get_move_reward.argtypes = [ctypes.c_ulonglong,   # white board
                                  ctypes.c_ulonglong,   # black board
                                  ctypes.c_bool,   # white engine
                                  ctypes.c_int32,   # max depth
                                  ctypes.POINTER(ctypes.c_int32)   # stats
                                  ]
    lib = ctypes.cdll.LoadLibrary(__file__.replace("__init__.py", "accelerator.so"))

    f_place_piece = lib.cpp_place_piece
    f_place_piece.argtypes = [ndpointer(ctypes.c_int32, flags="C_CONTIGUOUS"),   # **board
                                ctypes.c_int32,   # pos
                                ctypes.c_int32,   # value
                                ]

    f_legal_moves_array = lib.cpp_legal_moves_array
    f_legal_moves_array.argtypes = [ndpointer(ctypes.c_int32, flags="C_CONTIGUOUS"),   # **board
                                      ctypes.c_bool,   # reversed
                                      ]

except Exception as e:
    print(e)


def place_piece(board: np.array, pos: int, value: int) -> np.array:
    arr: np.array = deepcopy(board)
    f_place_piece(arr, pos, value)
    return arr


def legal_moves_array(board: np.array, color: int) -> np.array:
    return f_legal_moves_array(deepcopy(board), True if color == COLOR_BLACK else False)
```

CPP_engine/OthelloEngine.cpp

Implémentation première approche

```
#include <iostream>

#define HASHTABLE_CAPACITY 10000000

using bitboard = unsigned long long int;

/* In the tree search, engine is always represented by the black board ,and the opponent by the white board */

const int not_found_table = -100;

const bitboard POWER_2[64] = {

9223372036854775808,4611686018427387904,2305843009213693952,1152921504606846976,576460752303423488,288230376151711744,1
44115188075855872,72057594037927936,36028797018963968,18014398509481984,9007199254740992,4503599627370496,2251799813685
248,1125899906842624,562949953421312,281474976710656,140737488355328,70368744177664,35184372088832,17592186044416,87960
93022208,4398046511104,2199023255552,1099511627776,549755813888,274877906944,137438953472,68719476736,34359738368,17179
869184,8589934592,4294967296,2147483648,1073741824,536870912,268435456,134217728,67108864,33554432,16777216,8388608,419
4304,2097152,1048576,524288,262144,131072,65536,32768,16384,8192,4096,2048,1024,512,256,128,64,32,16,8,4,2,1,
};

/* Utils functions */

int min(int a, int b) {
    if (a < b) {
        return a;
    } else {
        return b;
    }
}

int max(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}

/* Game functions */
```

```
void place_piece(bitboard *player_board, bitboard *opponent_board, int pos) {

    bitboard o_board = *opponent_board;
    bitboard p_board = *player_board;
    bitboard opponent_edit;
    bitboard player_edit;

    p_board |= POWER_2[pos]; // place piece

    // edit every 8 possible directions

    int n;
    int i;
    int j;
    bool opponent_meet;
    int d;
    bitboard p;

    int list_n[8] = {pos / 8, // ^
                     pos % 8, // <-
                     min(pos / 8, 7 - (pos % 8)), // ^>
                     min(pos / 8, pos % 8), // <^
                     7 - (pos / 8), // V
                     min(7 - (pos / 8), 7 - (pos % 8)), // V>
                     min(7 - (pos / 8), pos % 8),  // <V
                     7 - (pos % 8) // ->
    };

    int list_d[8] = {-8, -1, -7, -9, 8, 9, 7, 1};

    for (j = 0; j < 8; j++) {

        opponent_edit = o_board;
        player_edit = p_board;

        n = list_n[j];
        d = list_d[j];

        opponent_meet = false;
```

```
        for (i = 1; i < (n + 1); i++) {

            p = POWER_2[pos + (i * d)];

            if (opponent_meet) {
                if ((p_board & p) == 0) {
                    if ((o_board & p) == 0) {
                        break;
                    } else {
                        opponent_edit &= ~p;
                        player_edit |= p;
                    }
                } else {
                    opponent_edit &= ~POWER_2[pos + d];
                    player_edit |= POWER_2[pos + d];

                    o_board = opponent_edit;
                    p_board = player_edit;
                    break;
                }
            } else {
                if ((o_board & p) != 0) {
                    opponent_meet = true;
                } else {
                    break;
                }
            }
        }
    }

    *player_board = p_board;
    *opponent_board = o_board;
}

bool is_legal_move(bitboard player_board, bitboard opponent_board, int pos) {

    // check if the case is empty
    if (((player_board | opponent_board) & POWER_2[pos])) {
        return false;
    }
```

```
int n;
int i;
int j;
bool opponent_meet;
int d;
bitboard p;

// check every 8 possible directions
int list_n[8] = {pos / 8, // ^
                 pos % 8, // <-
                 min(pos / 8, 7 - (pos % 8)), // ^>
                 min(pos / 8, pos % 8), // <^
                 7 - (pos / 8), // V
                 min(7 - (pos / 8), 7 - (pos % 8)), // V>
                 min(7 - (pos / 8), pos % 8),  // <V
                 7 - (pos % 8) // ->
};

int list_d[8] = {-8, -1, -7, -9, 8, 9, 7, 1};

for (j = 0; j < 8; j++) {

    n = list_n[j];
    d = list_d[j];

    opponent_meet = false;

    for (i = 1; i < (n + 1); i++) {

        p = POWER_2[pos + (i * d)];

        if (opponent_meet) {
            // If there is an empty case
            if ((player_board & p) == 0) {
                // If there is no opponent piece
                if ((opponent_board & p) == 0) {
                    break;
                }
            } else {
```

```
                    // if there is a player piece
                    return true;
                }
            } else {
                // If the first piece upward if an opponent piece
                if ((opponent_board & p) != 0) {
                    opponent_meet = true;
                } else {
                    break;
                }
            }
        }
    }

    return false;
}


bitboard get_all_legal_moves(bitboard player_board, bitboard opponent_board) {
    bitboard legal_move = 0;

    for (int i = 0; i < 64; i++) {
        if (is_legal_move(player_board, opponent_board, i)) {
            legal_move += POWER_2[i];
        }
    }

    return legal_move;
}


/* Engine functions */

class HashTable {
private:
    struct Entry {
        bitboard white_board;
        bitboard black_board;
        int reward;
    };
```

```
    unsigned int len = 0;

    Entry *table = new Entry[HASHTABLE_CAPACITY];

    unsigned int index(bitboard wb, bitboard bb) const {
        for (int i = 0; i < len; i++) {
            if (table[i].white_board == wb && table[i].black_board == bb) {
                return i + 1; // index + 1
            }
        }
        return 0; // 0 if not founded
    };

public:
    void add(bitboard wb, bitboard bb, int r) {
        if (len < HASHTABLE_CAPACITY) {
            table[len].white_board = wb;
            table[len].black_board = bb;
            table[len].reward = r;
            len += 1;
        }
    };

    int get(bitboard wb, bitboard bb) {
        unsigned int ind = index(wb, bb);
        if (ind == 0) {
            return not_found_table;
        } else {
            return table[ind - 1].reward;
        }
    }
};

int board_evaluate(bitboard board) {
    int count = 0;

    while (board) {
        count += board & 1;
        board >>= 1;
```

```
    }
    return count;
}


int explore_node(bitboard white_board, bitboard black_board, int depth, int max_depth, int *stats, HashTable H) {

    if (depth == max_depth) {
        return board_evaluate(black_board);
    }

    bitboard legal_moves;
    bitboard temp_w_board;
    bitboard temp_b_board;
    int reward;
    int temp_reward;
    int i;
    int hash_reward;

    hash_reward = H.get(white_board, black_board);

    if (hash_reward != not_found_table) {
        return hash_reward;
    }

    // Player turn
    if (depth % 2 == 1) {

        reward = 64; // impossible reward to do the min

        // get every possible moves
        legal_moves = get_all_legal_moves(white_board, black_board);

        for (i = 0; i < 64; i++) {

            if ((legal_moves & POWER_2[i]) != 0) {
                temp_w_board = white_board;
                temp_b_board = black_board;

                place_piece(&temp_w_board, &temp_b_board, i);
```

```
                *stats += 1;

                temp_reward = explore_node(temp_w_board, temp_b_board, depth + 1, max_depth, stats, H);

                reward = min(reward, temp_reward);
            }
        }

        H.add(white_board, black_board, temp_reward);

        return reward;
    } else {

        reward = 0; // impossible reward to do the max

        // get every possible moves
        legal_moves = get_all_legal_moves(black_board, white_board);

        for (i = 0; i < 64; i++) {

            if ((legal_moves & POWER_2[i]) != 0) {
                temp_w_board = white_board;
                temp_b_board = black_board;

                *stats += 1;

                place_piece(&temp_b_board, &temp_w_board, i);

                temp_reward = explore_node(temp_w_board, temp_b_board, depth + 1, max_depth, stats, H);

                reward = max(reward, temp_reward);
            }
        }

        H.add(white_board, black_board, temp_reward);

        return reward;
    }
}
```

```
int get_best_move(bitboard white_board, bitboard black_board, bool white_engine, int max_depth, int *stats) {
    bitboard wb;
    bitboard bb;

    if (white_engine) {
        wb = black_board;
        bb = white_board;
    } else {
        wb = white_board;
        bb = black_board;
    }

    int best_move = -11;
    int best_reward = -1;
    int temp_reward;
    int i;

    bitboard legal_moves;
    bitboard temp_w_board;
    bitboard temp_b_board;

    // create the hashtable
    HashTable H;

    legal_moves = get_all_legal_moves(bb, wb);

    for (i = 0; i < 64; i++) {

        if ((legal_moves & POWER_2[i]) != 0) {
            temp_w_board = wb;
            temp_b_board = bb;

            place_piece(&temp_b_board, &temp_w_board, i);

            temp_reward = explore_node(temp_w_board, temp_b_board, 1, max_depth, stats, H);
            if (temp_reward > best_reward) {
                best_reward = temp_reward;
```

```
                best_move = i;
            }
        }
    }

    return best_move;
}

int get_move_reward(bitboard white_board, bitboard black_board, bool white_engine, int max_depth, int *stats) {
    bitboard wb;
    bitboard bb;

    if (white_engine) {
        wb = black_board;
        bb = white_board;
    } else {
        wb = white_board;
        bb = black_board;
    }

    HashTable H;

    return explore_node(wb, bb, 0, max_depth, stats, H);
}

/* Export function */
extern "C" {
int cpp_get_move_reward(bitboard white_board, bitboard black_board, bool white_engine, int max_depth, int *stats) {
    return get_move_reward(white_board, black_board, white_engine, max_depth, stats);
}
}

extern "C" {
int cpp_get_best_move(bitboard white_board, bitboard black_board, bool white_engine, int max_depth, int *stats) {
    return get_best_move(white_board, black_board, white_engine, max_depth, stats);
}
}
```

static/board.css

Implémentation du jeu

```css
.main_div {
    display: flex;

    justify-content: center;
}


.board {
    margin-top: 2em;

    width: 55em;

    aspect-ratio: 1 / 1;

    background: white;
    border: 3px solid black;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.3);

    display: flex;
    flex-direction: column;

}

.board_line {
    height: 12.5%;
    width: 100%;
    display: flex;
    flex-direction: row;
}

.board_case {
    border-right: 1px solid #e5e5e5;
    border-top: 1px solid #e5e5e5;
    height: 100%;
    width: 12.5%;

    display: flex;
    justify-content: center;
    align-items: center;
}
```

```
.black_piece {
    width: 90%;
    height: 90%;

    background: black;
    border-radius: 50%;

    box-shadow: 0 0 5px rgba(0, 0, 0, 0.3);
}

.white_piece {
    width: 90%;
    height: 90%;

    background: white;

    border: 1px solid black;
    border-radius: 50%;

    box-shadow: 0 0 5px rgba(0,  0, 0, 0.3);
}

.possible_move_white {
    width: 90%;
    height: 90%;

    border: 1px solid transparent;
    border-radius: 50%;
    background-color: #3E79DA22;

    transition: 0.25s;
}

.possible_move_black {
    width: 90%;
    height: 90%;

    border: 1px solid transparent;
    border-radius: 50%;
```

```
    background-color: #00000022;

    transition: 0.5s;
}

.possible_move_white:hover {
    width: 90%;
    height: 90%;

    border: 1px solid transparent;
    border-radius: 50%;
    background-color: #3E79DA4C;

    transition: 0.5s;
}

.possible_move_black:hover {
    width: 90%;
    height: 90%;

    border: 1px solid transparent;
    border-radius: 50%;
    background-color: #0000004C;

    transition: 0.25s;
}

.possible_move_none {
    width: 90%;
    height: 90%;

    border: 1px solid transparent;
    border-radius: 50%;
}
```

static/game.js

Implémentation du jeu

```
const chart_data = [{"move": 0, "value": 50},]

const game_chart_config = {
    type: "line", data: {
        labels: chart_data.map(row => row.move), datasets: [{
            data: chart_data.map(row => row.value),
            pointRadius: 0,
            fill: true,
            cubicInterpolationMode: 'monotone',
            borderColor: "#00000000",
            backgroundColor: "rgba(0,0,0,0.52)",
        }]
    }, yAxes: [{
        display: true, ticks: {
            beginAtZero: true, steps: 10, stepValue: 5, max: 100, min: 0,
        },

    }], options: {
        responsive: true, maintainAspectRatio: false,

        y: {suggestedMin: 0, suggestedMax: 100}, animation: {duration: 0}, scales: {
            x: {
                grid: {drawOnChartArea: false}, ticks: {color: "#000000"}
            }, y: {
                grid: {drawOnChartArea: false}, ticks: {color: "#000000"}
            },

        }, scaleFontColor: "#000000", // hide dataset label
        plugins: {
            legend: {
                position: 'top', display: false,
            }, title: {
                display: false, text: 'Chart.js Line Chart',
            },
        },
    }
}

const game_old = new Chart(document.getElementById('score_chart'), game_chart_config)
```

```
let advice = true;

/*
All function for the game
*/
function plot_board(board, player) {
    /**
     This function plot the board depending on the board given in param
     this script only support 8*8 game
     @param board - a 2d array that contain board information
     */
    let board_string = "";

    for (let y = 0; y < 8; y++) {
        // start a new line in the board
        board_string += '<div class="board_line">'

        for (let x = 0; x < 8; x++) {
            // start a new row in the line
            board_string += '<div class="board_case">'

            // for black pieces
            if (board[y][x] === PIECE_BLACK) {
                board_string += '<div class="black_piece"></div>'

            }
            // for white pieces
            else if (board[y][x] === PIECE_WHITE) {
                board_string += '<div class="white_piece"></div>'

            }
            // for possible moves
            else if (board[y][x] === PIECE_MOVE) {
                if (!advice) {
                    board_string += `<div class="possible_move_none" id="${x}_${y}" onclick="play(${x},${y})"></div>`
                } else if (player === PIECE_BLACK && !engine_color) {
                    board_string += `<div class="possible_move_black" id="${x}_${y}" onclick="play(${x},${y})"></div>`
                } else if (player === PIECE_WHITE && engine_color) {
                    board_string += `<div class="possible_move_white" id="${x}_${y}" onclick="play(${x},${y})"></div>`
                }
```

```
            }
            board_string += '</div>'
        }
        board_string += '</div>'
    }
    document.getElementById("board").innerHTML = board_string;
}


function play(x, y) {
    /**
     * This function place piece i the board
     * @param x - the x coordinate of the move
     * @param y - the y coordinate of the move
     * */
    fetch(API_ENGINE_MOVE, {
        method: "POST",
        credentials: "include",
        headers: {
            "Content-Type": "application/json",
            "Access-Control-Allow-Credentials": true,
        },
        body: JSON.stringify({"x": x, "y": y})
    }).then(r => {
        return r;
    })
}


function update_score(score_black, score_white) {
    /**
     This function update the score of the game
     @param score_black - the number of black pieces on the board
     @param score - white: the number of white pieces on the board
     @return: null
     */
    // update the score
    document.getElementById("score_black").innerHTML = score_black
    document.getElementById("score_white").innerHTML = score_white
```

```
    // update the bar
    document.getElementById("bar_black").style.width = (score_black / (score_black + score_white) * 100) + "%"
    document.getElementById("bar_white").style.width = (score_white / (score_black + score_white) * 100) + "%"
}


function update_history(history) {
    /**
     * This function update the game history
     * @param history - The history of the game as json
     */
    let history_string = ""
    let last = "0"
    let piece
    let style_bolder = "font-weight: bold;"
    let style_italic = ""

    let y_chart = []
    let x_chart = []

    for (let i in history) {

        history_string += '<li class="move_li">'

        // history black
        if (history[i]["score"][0] === history[last]["score"][0]) {
            history_string += '<span class="move_info_score_left"> </span>'
        } else if (history[i]["score"][0] > history[last]["score"][0]) {
            history_string += `<span class="move_info_score_left" style="${style_bolder}"> +${history[i]["score"][0] -
history[last]["score"][0]} </span>`
        } else {
            history_string += `<span class="move_info_score_left" style="${style_italic}"> ${history[i]["score"][0] -
history[last]["score"][0]} </span>`
        }

        // move
        if (i === "0") {
            history_string += '<span class="move_info"> Middle </span>'
        } else {
            if (history[i]["player"] === PIECE_BLACK) {
```

```
                piece = "Black"
            } else {
                piece = " White"
            }
            history_string += `<span class="move_info"> ${piece}: ${history[i]["move"][0]} |
${history[i]["move"][1]}</span>`
        }


        // history White
        if (history[i]["score"][1] === history[last]["score"][1]) {
            history_string += '<span class="move_info_score_right"> </span>'
        } else if (history[i]["score"][1] > history[last]["score"][1]) {
            history_string += `<span class="move_info_score_right" style="${style_bolder}"> +${history[i]["score"][1] -
history[last]["score"][1]} </span>`
        } else {
            history_string += `<span class="move_info_score_right" style="${style_italic}"> ${history[i]["score"][1] -
history[last]["score"][1]} </span>`
        }

        history_string += "</li>"

        last = i


        y_chart.push(history[i]["score"][0] / (history[i]["score"][1] + history[i]["score"][0]) * 100);
        x_chart.push(i);
    }

    document.getElementById("history").innerHTML = history_string

    // update the chart
    game_old.data.labels = x_chart;
    game_old.data.datasets[0].data = y_chart;
    game_old.update();
}


function update_advice() {
    /**
     * This function update the visibility of advice
```

```
     */
    advice = !advice; // update the advice statement
    // update the button
    if (advice) {
        document.getElementById("button_advice").innerHTML = "Hide <i>advice</i>"
    } else {
        document.getElementById("button_advice").innerHTML = "Show <i>advice</i>"
    }

    const source_update = new EventSource(STREAM_API_BOARD);

    source_update.onmessage = function (event) {
        const stream_data = JSON.parse(JSON.stringify(event.data))
        let dataReceived = JSON.parse(stream_data)

        plot_board(dataReceived["move_board"], dataReceived["player"]);

        source_update.close();
    };
}


function change_engine_color() {
    /**This function change the color of the engine
     * */
    engine_color = !engine_color;

    fetch(API_SETTING, {
        method: "post",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({"setting": "change_engine_color"})
    }).then(r => {
        return r
    })

    // engine color == black
    if (engine_color) {
        document.getElementById("button_play_as").innerHTML = "Play as <i>black</i>"
    } else {
        document.getElementById("button_play_as").innerHTML = "Play as <i>white</i>"
```

```
    }

}


function restart() {
    /** This function restart the current game
     * */
    fetch(API_SETTING, {
        method: "post",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({"setting": "restart"})
    }).then(r => {
        return r;
    })
}

// stream the board
const source = new EventSource(STREAM_API_BOARD);

source.onmessage = function (event) {
    const stream_data = JSON.parse(JSON.stringify(event.data))
    let dataReceived = JSON.parse(stream_data)

    update_score(dataReceived["score_black"], dataReceived["score_white"]);
    plot_board(dataReceived["move_board"], dataReceived["player"]);
    update_history(dataReceived["history"])

    // update who plays
    if (dataReceived["player"] === PIECE_BLACK) {
        document.getElementById("info_title_left").innerHTML = "<b style='text-decoration: underline;'>Black</b>"
        document.getElementById("info_title_right").innerHTML = "White"
    } else {
        document.getElementById("info_title_left").innerHTML = "Black"
        document.getElementById("info_title_right").innerHTML = "<b style='text-decoration: underline;'>White</b>"
    }
};
```

static/panel.css

Implémentation du jeu

```css
.panel {
    background: white;
    border: 3px solid black;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.3);

    margin-left: 2.5%;
    margin-top: 2em;

    width: 32.5%;
}

.title_img_icon {
    display: inline-block;
    height: 65px;
    margin: 5px;
}

.title_text {
    margin-left: 10px;
    margin-bottom: 5px;
    margin-top: 5px;

    position: absolute;

    font-family: Roboto, serif;
    font-size: 30px;

    line-height: 65px;
    vertical-align: middle;

    white-space: nowrap;
}

/*
##### Score ######
*/
.score {
    margin: 2.5em 5px 5px;
    display: flex;
    flex-direction: column;
```

```
    transition: 0.5s;

    height: 3em;

    overflow: hidden;
}

.score:hover {
    height: 15em;
    transition: 0.5s;
    transition-delay: 0.5s;
}

.score_title_span {
    font-family: Roboto, serif;
    font-size: 17px;
}

#score_title_left {
    float: left;
    margin-left: 2px;
}

#score_title_right {
    float: right;
    margin-right: 2px;
}

.score_bar {
    background: white;
    border: 2px solid black;
    height: 1.5em;

    display: flex;
    flex-direction: row;
    justify-content: center;
}

.score_bar_black {
```

```
    width: 50%;
    height: 1.5em;
    background-color: black;
}

.score_bar_white {
    width: 50%;
    height: 1.5em;
    background-color: white;
}

.score_chart {
    background-color: white;
    height: 100%;
    border: 2px solid black;
    border-top: 1px solid transparent;
    transition: 0.25s;
}

/*
##### Game information #####
*/

.game_info {
    margin: 2.5em 5px 5px;
    height: 75%;

    transition: 0.5s;

    border: 3px solid black;
    background-color: white;

    display: flex;
    justify-content: space-between;
    flex-direction: column;
}


/* player */
.info_player {
```

```
    float: top;
    padding: 0.5em;
    border-bottom: 3px solid black;
}

.info_title_span {
    font-family: Roboto, serif;
    font-size: 17px;
}

#info_title_left {
    float: left;
}

#info_title_right {
    float: right;
}

/* History */

.info_history {
    height: 100%;
    display: flex;
    flex-direction: column;
    align-content: center;
}

/* Menu */

.info_menu {
    border-top: 3px solid black;
    padding: 0.25em;
}

.info_button {
    border: 2px solid black;

    font-family: Roboto, serif;
    font-size: 17px;
```

```
    padding: 0.25em;
    width: 20%;

    text-align: center;

    transition: 0.25s;
}

.info_button:hover {
    border: 2px solid black;

    font-family: Roboto, serif;
    font-weight: bold;
    font-size: 17px;

    padding: 0.25em;

    box-shadow: 0 0 10px rgba(0, 0, 0, 0.3);

    transition: 0.25s;
}

#button_restart {
    float: left;
}

#button_advice {
    float: right;
}

/* move information */
ul {
    list-style: none;
    padding: 0;

    display: block;
    margin-left: 1em;
    margin-right: 0.5em;
}
```

```css
.move_li {
    text-align: center;
    font-family: Roboto, serif;
    font-size: 17px;
    margin-top: 5px;
    margin-bottom: 5px;
}

.scroll_menu {
    max-height: 34em;
    height: auto;
    overflow-y: auto;
    text-align: center;
}

.scroll_menu::-webkit-scrollbar-track {
    background-color: transparent;

}

.scroll_menu::-webkit-scrollbar {
    width: 3px;
    background-color: transparent;
}

.scroll_menu::-webkit-scrollbar-thumb {
    background-color: #000000;
}

.move_info_score_left {
    display: inline-block;
    width: 50px;
    min-width: 50px;
    float: left;
}

.move_info_score_right {
    min-width: 50px;
    width: 50px;
    float: right;
```

```
}

.move_info {
    text-align: center;
    min-width: 100px;
    width: 100px;
}

.info_menu {
    display: flex;
    flex-direction: row;
    justify-content: space-between;
}
```

templates/board.html

Implémentation du jeu

```
<link rel="stylesheet" href="{{ url_for('static', filename='board.css') }}">

<!-- Board for the game -->
<div>
    <div class="board" id="board">
        <div class="board_line">
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
        </div>
        <div class="board_line">
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
        </div>
        <div class="board_line">
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
        </div>
        <div class="board_line">
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
```

```
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
</div>
<div class="board_line">
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
</div>
<div class="board_line">
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
</div>
<div class="board_line">
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
</div>
<div class="board_line">
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
    <div class="board_case"></div>
```

```
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
            <div class="board_case"></div>
        </div>
    </div>
</div>
```

templates/index.html

Implémentation du jeu

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Othello AI</title>
    <link rel="icon" type="image/gif/png" href="{{ url_for('static', filename='images/oi_icon.ico') }}">

    <script src="{{ url_for('static', filename='chart.js') }}"></script>

    <script type="text/javascript">
        /*Imports*/
        const PIECE_EMPTY = 0;
        const PIECE_BLACK = 2;
        const PIECE_WHITE = 1;
        const PIECE_MOVE = 5;

        const SCORE_WHITE_WIN = 1;
        const SCORE_BLACK_WIN = 2;

        const API_ENGINE_MOVE   = "/api/move/";
        const API_SETTING = "/api/setting/";
        const STREAM_API_BOARD = "/api/game-stream/";

        let engine_color = {{ 'true' if ENGINE_BLACK == 2 else "false" }};
    </script>

</head>

<body>

<div class="main_div">

    <!-- Board for the game -->
    {% include "board.html" %}

    <!-- Panel for the game -->
    <link rel="stylesheet" href="{{ url_for('static', filename='panel.css') }}">

    <div class="panel">
        <div class="title">
```

```
        <img class="title_img_icon" src="{{ url_for('.static', filename='images/logo_png.png') }}" alt="Game">
    </div>

    <div class="score">
        <div class="score_title">
            <span class="score_title_span" id="score_title_left">Black <b id="score_black">50</b></span>
            <span class="score_title_span" id="score_title_right"><b id="score_white">50</b> White</span>
        </div>

        <div class="score_bar">
            <div class="score_bar_black" id="bar_black"></div>
            <div class="score_bar_white" id="bar_white"></div>
        </div>

        <div class="score_chart">
            <canvas id="score_chart" class="canvas_chart"></canvas>
        </div>
    </div>
    <!-- Information of the game -->
    <div class="game_info">
        <div class="info_player">
            <span class="info_title_span" id="info_title_left">Black</span>
            <span class="info_title_span" id="info_title_right">White</span>
        </div>
        <div class="info_history">
            <ul class="scroll_menu" id="history">

                <li class="move_li">
                    <span class="move_info_score_left">+10</span>
                    <span class="move_info">Black : AB</span>
                    <span class="move_info_score_right">-10</span>
                </li>

            </ul>
        </div>
        <div class="info_menu">
            <div class="info_button" id="button_restart" onclick="restart()"> Restart <i>Game</i></div>
            <div class="info_button" id="button_play_as" onclick="change_engine_color()"> Play as
<i>black</i></div>
            <div class="info_button" id="button_advice" onclick="update_advice()"> Hide <i>advice</i></div>
```

```
            </div>
        </div>
    </div>

</div>

<script src="{{ url_for('static', filename='game.js') }}"></script>
</body>
```

Tensrflow_engine/__init__.py

Résolution avec réseaux de neurones

```python
import time
from copy import deepcopy
import ctypes
import random

from keras.layers import Activation
from keras.layers import BatchNormalization
from keras.layers import Conv2D
from keras.layers import Dense
from keras.layers import Flatten
from keras.layers import Lambda
from keras.layers import Input
from keras.layers import LeakyReLU
from keras.layers import Reshape
from keras.models import Model
from keras.models import load_model
from keras.utils.vis_utils import plot_model
import numpy as np
from numpy.ctypeslib import ndpointer
import tensorflow as tf

COLOR_WHITE: int = 1
COLOR_BLACK: int = 2

GPU_USAGE: bool = False

if len(tf.config.list_physical_devices('GPU')) == 0 or not GPU_USAGE:
    DEVICE: str = "/cpu:0"
else:
    DEVICE: str = "/gpu:0"

print(f"Using {DEVICE}")


def board_to_input(board: list[list[int]], color: int) -> np.array:
    return np.array([[1.0 if x == color else (0.5 if x == 0 else 0.0) for x in y] for y in board])


def noise(arr: np.array, alpha: float) -> np.array:
    return np.random.uniform(-alpha, alpha, arr.shape) + arr
```

```python
def count_eval(board: np.array, color: int = COLOR_BLACK) -> float:
    t = np.count_nonzero(board != 0)
    p = np.count_nonzero(board == color)
    return p / t if t < 64 else (0 if p < 32 else 1)


class GlobalNetwork:
    def __init__(self,
                 device: str = DEVICE,
                 train: bool = False,
                 prob_random: float = 0.5,
                 count_evaluation: bool = False,
                 noise_alpha: float = 0.03):
        super().__init__()

        inputs = Input(shape=(8, 8, 1))

        x = Lambda(lambda val: val)(inputs)

        for i in range(7):
            x = self.residual_block(x)

        policy_out = self.policy_head(x)
        value_out = self.value_head(x)

        self.network = Model(inputs=inputs, outputs=[policy_out, value_out])

        # self.network.summary()

        self.network.compile(loss='mse', optimizer='adam', metrics=["accuracy", "mse"], run_eagerly=True)

        self.device: str = device
        self.train_mode: bool = train
        self.prob_random: float = prob_random
        self.noise_alpha: float = noise_alpha
        self.count_evaluation: bool = count_evaluation

    @staticmethod
```

```
def residual_block(x):
    original = x
    x = Conv2D(filters=64,
               kernel_size=(3, 3),
               padding="same")(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)

    x = Conv2D(filters=64,
               kernel_size=(3, 3),
               padding="same")(x)
    x = BatchNormalization()(x)

    x += original

    return LeakyReLU()(x)

@staticmethod
def policy_head(x):
    x = Conv2D(filters=2,
               kernel_size=(1, 1),
               padding="same")(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)

    x = Dense(1)(x)

    x = Reshape((64,))(x)
    x = Activation("sigmoid")(x)

    return x

@staticmethod
def value_head(x):
    x = Conv2D(filters=1,
               kernel_size=(1, 1),
               padding="same")(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
```

```
    x = Flatten()(x)

    x = Dense(1)(x)
    x = Activation("sigmoid")(x)

    return x

def __call__(self, board: np.array, color: int, *args, **kwargs) -> np.array:
    if not self.train_mode:

        b = noise(board_to_input(board, COLOR_BLACK), self.noise_alpha)

        with tf.device(self.device):
            policy, value = self.network(np.array([b]), *args, training=False, **kwargs)
            policy = policy[0].numpy()
            value = float(value[0][0])

        return policy, count_eval(board=board, color=color) if self.count_evaluation else value

    else:
        if random.random() < self.prob_random:
            return np.random.random((64,)), count_eval(board=board,
                                                       color=color) if self.count_evaluation else random.random()
        else:
            b = noise(board_to_input(board, COLOR_BLACK), self.noise_alpha)

            with tf.device(self.device):
                policy, value = self.network(np.array([b]), *args, training=False, **kwargs)
                policy = policy[0].numpy()
                value = float(value[0][0])

            return policy, count_eval(board=board, color=color) if self.count_evaluation else value

def save_model(self, path: str) -> None:
    self.network.save(path)

def load_model(self, path: str) -> None:
    new = GlobalNetwork()
    new.network = load_model(path)
    self.network.set_weights(new.network.get_weights())
```

```
    def train(self, x, y, *args, device: str = DEVICE, **kwargs) -> None:
        with tf.device(device):
            self.network.fit(x, y, *args, **kwargs)


class TensorflowOthelloEngine:
    def __init__(self,
                 max_depth: int,
                 device: str = DEVICE,
                 train: bool = False,
                 rate_stop_exploration: float = 0.30,
                 prob_random: float = 0.5,
                 max_branch: int = 64,
                 max_search_time: float = 100.0,
                 count_evaluation: bool = False,
                 base_elo: float = 1000,
                 noise_alpha: float = 0.03):
        self.global_network: GlobalNetwork = GlobalNetwork(device=device,
                                                           train=train,
                                                           prob_random=prob_random,
                                                           noise_alpha=noise_alpha,
                                                           count_evaluation=count_evaluation)

        self.max_depth: int = max_depth
        self.max_branch: int = max_branch
        self.max_search_time: float = max_search_time
        self.count_evaluation: bool = count_evaluation
        self.rate_stop_exploration: float = rate_stop_exploration

        self.elo_score: float = base_elo

    @property
    def get_elo(self) -> float:
        return self.elo_score

    def evaluation(self, board) -> tuple[float, float]:
        _, value = self.global_network(board, COLOR_BLACK)
        return value, count_eval(board, COLOR_BLACK)
```

```python
def explore_node(self, board: np.array, depth: int) -> tuple[float, float]:
    if depth == self.max_depth:
        return self.evaluation(board)

    # opponent is playing
    if depth % 2 == 1:
        policy, value = self.global_network(board, COLOR_WHITE)

        if (1 - value) < self.rate_stop_exploration:
            return (1 - value), count_eval(board, COLOR_BLACK)
        else:
            mask = legal_moves_array(board, COLOR_WHITE).flatten()
            explore = mask * policy
            best_indices: int = int(np.argmax(explore))

            if explore[best_indices] == 0.0:
                return self.evaluation(board)

            else:
                new_board = place_piece(board, best_indices, COLOR_WHITE)
                return self.explore_node(new_board, depth + 1)

    # engine is playing
    else:
        policy, value = self.global_network(board, COLOR_BLACK)

        if value < self.rate_stop_exploration:
            return value, count_eval(board, COLOR_BLACK)
        else:
            mask = legal_moves_array(board, COLOR_BLACK).flatten()
            explore = mask * policy
            best_indices: int = int(np.argmax(explore))

            if explore[best_indices] == 0.0:
                return self.evaluation(board)

            else:
                new_board = place_piece(board, best_indices, COLOR_BLACK)
                return self.explore_node(new_board, depth + 1)
```

```
def __call__(self,
             board: list[list[int]],
             engine_value: int,
             *args,
             return_reward: bool = False,
             **kwargs) -> tuple[int, int] | tuple[int, int, float]:
    array_board = np.array(
        [[COLOR_BLACK if x == engine_value else (0 if x == 0 else COLOR_WHITE) for x in y] for y in board],
        dtype=np.int32)

    mask = legal_moves_array(array_board, COLOR_BLACK).flatten()
    policy, value = self.global_network(array_board, COLOR_BLACK)
    policy = policy * mask

    best_move = tf.argsort(mask.flatten(), direction='DESCENDING').numpy()[0]
    best_indices = tf.argsort(policy, direction='DESCENDING')
    max_return, max_c = 0.0, 0.0

    i: int = 0
    t0: float = time.time()

    while (i < self.max_branch) and (policy[best_indices[i]] > 0.0) and (time.time() - t0 < self.max_search_time):

        new_board = place_piece(array_board, best_indices[i], COLOR_BLACK)

        r, c = self.explore_node(new_board, 1)

        if r > max_return:
            best_move = best_indices.numpy()[i]
            max_return = r

        max_c = max(c, max_c)
        i += 1

    if return_reward:
        return int(best_move % 8), int(best_move // 8), max_c
    else:
        return int(best_move % 8), int(best_move // 8)
```

```
try:
    lib = ctypes.cdll.LoadLibrary(__file__.replace("__init__.py", "accelerator.so"))

    f_place_piece = lib.cpp_place_piece
    f_place_piece.argtypes = [ndpointer(ctypes.c_int32, flags="C_CONTIGUOUS"),   # **board
                              ctypes.c_int32,  # pos
                              ctypes.c_int32,  # value
                              ]

    f_legal_moves_array = lib.cpp_legal_moves_array
    f_legal_moves_array.argtypes = [ndpointer(ctypes.c_int32, flags="C_CONTIGUOUS"),   # **board
                                    ctypes.c_bool,  # reversed
                                    ]
except Exception as e:
    raise e


def place_piece(board: np.array, pos: int, value: int) -> np.array:
    arr: np.array = deepcopy(board)

    f_place_piece(arr, pos, value)

    return arr


def legal_moves_array(board: np.array, color: int) -> np.array:
    arr: np.array = deepcopy(board)

    if color == COLOR_BLACK:
        f_legal_moves_array(arr, True)
    else:
        f_legal_moves_array(arr, False)

    return arr


if __name__== '__main__':
    a = GlobalNetwork()
    # Pour obtenir certaines images de diapositives
    plot_model(a.network, to_file='model_plot_global.jpg', show_shapes=True, show_layer_names=True)
```

evaluate_model.py

Test du modèle

```python
import random
from copy import deepcopy
import os

if __name__!= '__main__':
    os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

import numpy as np

from cpp_engine import CppOthelloEngine
from othello import Othello
from othello import COLOR_WHITE
from othello import COLOR_BLACK
from othello import SCORE_NOT_FINISH
from othello import SCORE_CANT_WHITE
from othello import SCORE_CANT_BLACK
from tensorflow_engine import board_to_input
from tensorflow_engine import legal_moves_array
from tensorflow_engine import TensorflowOthelloEngine

MAX_DEPTH: int = 16

GAME_DEVICE: str = "/cpu:0"
LOAD_PREVIOUS_MODEL: bool = True
VERBOSE: bool = False

CPP: bool = True
NAME: str = "CPPEngine" if CPP else "TensorflowEngine 2"


def print_board(b: list[list[int]]):
    for line in b:
        for case in line:
            print(". " if case == 0 else ("W " if case == COLOR_WHITE else "B "), end="")
        print("\n", end="")
    print()


def softmax(b: np.array):
    total = np.sum(b)
```

```
    return b / total


def print_prob(b: list[list[int]], color: int) -> None:

    p = model1.global_network(b, 1)
    print(f"Probabilité of winning white: {p[1]}")
    p = model1.global_network(b, 2)
    print(f"Probabilité of winning black: {p[1]}")
    p = model1.global_network(b, color)
    print("Policy network prediction:")

    r = p[0]
    mask = legal_moves_array(np.array(b, dtype=np.int32), color).flatten() * r
    sm = np.array(mask / np.sum(mask))

    for y in range(8):
        for x in range(8):

            if sm[y * 8 + x] == 0.0:
                print(f" {'B' if b[y][x] == 2 else ('.' if b[y][x] == 0 else 'W')} ", end="")

            else:
                val = int(sm[y * 8 + x] * 100)
                sup = 3 - len(str(val))
                print(f" {val}{' ' * (sup - 1)}%", end="")

        print("\n")
    print("==================================")


if __name__== '__main__':
    model1 = TensorflowOthelloEngine(max_depth=MAX_DEPTH,
                                     device=GAME_DEVICE,
                                     count_evaluation=False )
    model1.global_network.load_model("global_net2.h5")

    if CPP:
        model2 = CppOthelloEngine(7, logs=False)
    else:
```

```
    model2 = TensorflowOthelloEngine(max_depth=MAX_DEPTH,
                                     device=GAME_DEVICE,
                                     count_evaluation=False,
                                     rate_stop_exploration=0.30)
    model2.global_network.load_model("global_net.h5")

game = Othello()
model1_color = random.choice([COLOR_BLACK, COLOR_WHITE])
model2_color = COLOR_BLACK if model1_color == COLOR_WHITE else COLOR_WHITE

print(f"TensorflowEngine: {'W' if model1_color == COLOR_WHITE else 'B'}")
print(f"{NAME}: {'B' if model2_color == COLOR_BLACK else 'W'}")

while game.game_statu in [SCORE_NOT_FINISH, SCORE_CANT_WHITE, SCORE_CANT_BLACK]:
    print("==================================")
    board = deepcopy(game.get_board)
    who_plays = game.who_plays

    print_board(board)
    s = game.get_score
    if model1_color == COLOR_WHITE:
        print(f"TensorflowEngine (W): {s[1]}    |   {NAME} (B): {s[0]}")
    else:
        print(f"TensorflowEngine (B): {s[0]}    |   {NAME} (W): {s[1]}")

    print_prob(board, who_plays)

    print(f"{'TensorflowEngine' if model1_color == who_plays else NAME} is playing...")

    if who_plays == model1_color:
        x, y = model1(board, model1_color)
        game.place_piece(x, y)

    else:
        x, y = model2(board, model2_color)
        game.place_piece(x, y)

print("Final board:")
print_board(game.get_board)
```

```
print("Final score:")
s = game.get_score
if model1_color == COLOR_WHITE:
    print(f"TensorflowEngine (W): {s[1]}")
    print(f"{NAME} (B):         {s[0]}")
else:
    print(f"TensorflowEngine (B): {s[0]}")
    print(f"{NAME} (W):         {s[1]}")
```

live_graph.py

Visualisation des performances

```
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np
import pandas as pd

matplotlib.use("tkagg")

fig, axs = plt.subplots(2, 2)

axs2 = axs[0][0].twinx()
axs3 = axs[0][1].twinx()

MA_LEN: int = 25


def moving_average(x, length):
    r = []

    for i in range(0, len(x)):
        a = x[max(0, i - length):i]
        r.append(np.sum(a) / min(i, length))

    return r


def plot(data):
    axs[0][0].clear()
    axs[1][0].clear()
    axs[0][1].clear()
    axs[1][1].clear()

    x = data["game played"].values

    # ax 1
    axs[0][0].clear()
    axs2.clear()

    l1 = axs[0][0].plot(x, data["model accuracy"].values, c="tab:blue", label="Accuracy")
    axs[0][0].plot(x, moving_average(data["model accuracy"].values, MA_LEN), c="black", alpha=0.25)
```

```
l2 = axs2.plot(x, data["model loss"].values, c="tab:orange", label="Loss")
axs2.plot(x, moving_average(data["model loss"].values, MA_LEN), c="black", alpha=0.25)

axs[0][0].set_title("Neural network accuracy/loss")
lt = l1 + l2
labs = [lab.get_label() for lab in lt]
axs[0][0].legend(lt, labs)
axs[0][0].set_ylim(0, 1.1)

# ax 2
axs[0][1].clear()
axs3.clear()

l1 = axs[0][1].plot(x, data["model accuracy"].values, c="tab:blue", label="Accuracy")
axs[0][1].plot(x, moving_average(data["model accuracy"].values, MA_LEN), c="black", alpha=0.25)

l2 = axs3.plot(x, data["model loss"].values, c="tab:orange", label="Loss")
axs3.plot(x, moving_average(data["model loss"].values, MA_LEN), c="black", alpha=0.25)

axs[0][1].set_title("Neural network accuracy/loss")
lt = l1 + l2
labs = [lab.get_label() for lab in lt]
axs[0][1].legend(lt, labs)

# ax 3
axs[1][0].clear()
d = data["elo rating"].values
d[702:] += 220
axs[1][0].step(x, d, c="tab:blue")
axs[1][0].set_title("Model Elo rating")

# ax 4
y = (data["score"].values * 64)

axs[1][1].clear()
axs[1][1].plot(x, [max(32, v) for v in y], c="tab:green", alpha=0.5)
axs[1][1].plot(x, [min(32, v) for v in y], c="tab:red", alpha=0.5)

axs[1][1].plot(x, moving_average(y, MA_LEN), c="black", alpha=0.75)
```

```
    axs[1][1].plot(x, [32 for _ in x], c="black", alpha=0.75)
    axs[1][1].set_title("Model score")
    axs[1][1].set_ylim(0, 64)


def animate(i):
    data = pd.read_csv("train_history.txt", encoding="utf-8", sep=";")
    plot(data)


if __name__== '__main__':
    plot(pd.read_csv("train_history.txt", encoding="utf-8", sep=";", ))
    ani = animation.FuncAnimation(fig, animate, interval=5000)
    plt.show()
```

main.py

Implémentation du jeu

```
"""
Toutes les fonctions principles qui permettent de jouer à l'Othello
"""
from copy import deepcopy
import json
import time
import random
from uuid import uuid4

from flask import Flask
from flask import request
from flask import Response
from flask import render_template
from flask import session
from flask import stream_with_context
import numpy as np

from othello import COLOR_BLACK
from othello import Othello
from cpp_engine import CppOthelloEngine
from tensorflow_engine import board_to_input
from tensorflow_engine import legal_moves_array
from tensorflow_engine import TensorflowOthelloEngine

ENGINE_NAME: str = "TF"  # CPP or TF

app = Flask(__name__)
app.secret_key = str(random.random()).replace(".", "")

GAMES: dict = {}  # Toutes les parties de chaque session
ENGINE_COLOR: dict = {}  # La couleur sur le plateau du model qui joue

CPP_ENGINE = CppOthelloEngine(n_thread=1,
                              logs=True,
                              max_depth=7)
TF_ENGINE = TensorflowOthelloEngine(max_depth=13,
                                    device="/cpu:0",
                                    train=False,
                                    max_branch=16,
                                    max_search_time=2.5,
```

```
                                    count_evaluation=False)
TF_ENGINE.global_network.load_model("global_net2.h5")

AI_STATS: bool = True


def ai_stats(board: list[list[int]], color: int) -> None:
    p = TF_ENGINE.global_network(board, 1)
    print(f"Probabilité of winning white: {p[1]}")
    p = TF_ENGINE.global_network(board, 2)
    print(f"Probabilité of winning black: {p[1]}")
    p = TF_ENGINE.global_network(board, color)

    print("Policy network prediction:")

    r = p[0]
    mask = legal_moves_array(np.array(board, dtype=np.int32), color).flatten() * r
    sm = np.array(mask / np.sum(mask))

    for y in range(8):
        for x in range(8):

            if sm[y * 8 + x] == 0.0:
                print(f" {'B' if board[y][x] == 2 else ('.' if board[y][x] == 0 else 'W')} ", end="")

            else:
                val = int(sm[y * 8 + x] * 100)
                sup = 3 - len(str(val))
                print(f" {val}{' ' * (sup - 1)}%", end="")

        print("\n")
    print("=================")


def reset_session():
    token: str = str(uuid4())
    session["token"]: str = token
    GAMES[token] = Othello()
    ENGINE_COLOR[token] = COLOR_BLACK
```

```
def game_stream():
    if "token" not in session or not session.get("token") in GAMES:
        reset_session()

    token = session["token"]
    session_game: Othello = GAMES[token]
    move_count: int = session_game.game_history.get_move_count - 1   # last player

    while True:

        session_game: Othello = GAMES[token]

        if move_count != session_game.game_history.get_move_count:
            if AI_STATS:
                try:
                    ai_stats(session_game.get_board, session_game.who_plays)
                except Exception as e:
                    print(e)

            move_count = session_game.game_history.get_move_count

            return_json: dict = {"board": session_game.get_board,
                                 "player": session_game.get_player}

            move_board = deepcopy(session_game.get_board)

            for m in session_game.get_possible_moves():
                move_board[m[1]][m[0]] = 5

            return_json["move_board"] = move_board

            score = session_game.get_score

            return_json["score_black"] = score[0]
            return_json["score_white"] = score[1]
            return_json["statu"] = session_game.game_statu

            # create a history
            return_json["history"] = {}
```

```
            i = 0
            for move in session_game.game_history:
                return_json["history"][str(i)] = {"move": move[1],
                                                  "board": move[0],
                                                  "score": [int(move[2][0]), int(move[2][1])],
                                                  "player": int(move[3])}
                i += 1

            yield f"data:{json.dumps(return_json)}\n\n"

        # engine move
        if session_game.get_player == ENGINE_COLOR[token]:
            if len(session_game.get_possible_moves()) > 0:
                engine_move(token, ENGINE_COLOR[token], ENGINE_NAME)

        time.sleep(0.01)


def engine_move(token: str, color: int, engine: str) -> None:
    if engine == "CPP":
        session_game: Othello = GAMES[token]

        x, y = CPP_ENGINE(board=deepcopy(session_game.get_board),
                          engine_value=color)

        if session_game.get_player == color:
            session_game.place_piece(x=x, y=y)

    elif engine == "TF":
        session_game: Othello = GAMES[token]

        x, y = TF_ENGINE(board=session_game.get_board,
                         engine_value=color)

        if session_game.get_player == color:
            session_game.place_piece(x=x, y=y)


@app.route("/api/move/", methods=["GET", "POST"])
def api_engine_move() -> str:
```

```
    if "token" not in session or not session.get("token") in GAMES:
        token: str = str(uuid4())
        session["token"]: str = token

        reset_session()

    # player move
    session_game: Othello = GAMES[session["token"]]

    if session_game.get_player != ENGINE_COLOR[session["token"]]:
        # get information given by the script
        data: dict = dict(request.get_json())

        x: int = data["x"]
        y: int = data["y"]

        session_game.place_piece(x=x, y=y)

    return ""


@app.route("/api/setting/", methods=["GET", "POST"])
def api_engine_setting() -> str:
    data: dict = dict(request.get_json())
    setting: str = data["setting"]

    token = session["token"]

    if setting == "change_engine_color":
        ENGINE_COLOR[token] = 1 if ENGINE_COLOR[token] == 2 else 2
        GAMES[token] = Othello()

    if setting == "restart":
        GAMES[token] = Othello()

    return ""


@app.route("/api/game-stream/", methods=["GET", "POST"])
def api_game_stream():
```

```
    return Response(stream_with_context(game_stream()), mimetype="text/event-stream")


@app.route("/", methods=["GET"])
def main() -> str:
    # generation des tokens pour la session
    if "token" not in session:
        reset_session()

    return render_template("index.html", ENGINE_BLACK=ENGINE_COLOR[session["token"]])


if __name__== '__main__':
    app.run(
        host="127.0.0.1",
        port=4000,
        debug=True
    )
```

othello.py

Implémentation du jeu

```python
from copy import deepcopy

COLOR_WHITE: int = 1
COLOR_BLACK: int = 2

SCORE_WIN_W: int = COLOR_WHITE
SCORE_WIN_B: int = COLOR_BLACK
SCORE_NOT_FINISH: int = 0
SCORE_TIE: int = 3
SCORE_CANT_WHITE: int = 11
SCORE_CANT_BLACK: int = 22


class Othello:
    def __init__(self) -> None:
        # crate the board
        self.board: list[list[int]] = [[0 for _ in range(8)] for _ in range(8)]
        self.board[3][3] = COLOR_WHITE
        self.board[4][4] = COLOR_WHITE
        self.board[4][3] = COLOR_BLACK
        self.board[3][4] = COLOR_BLACK

        # set the first player
        self.who_plays: int = COLOR_BLACK

        self.game_history = GameHistory()

        self.game_history.append(self.board, (), (2, 2), -1)

    # All information about the board
    def get_possible_moves(self, player=None) -> list[tuple[int, int]]:
        """
        This function return the coordinate of each possible moves (x, y)
        :return: list of tuple [(x, y), (x', y'), ... , (x'', y'')]
        """
        if player is None:
            p = self.who_plays
        else:
            p = player
```

```
    moves: [tuple] = []

    for y in range(8):
        for x in range(8):
            if self.board[y][x] == 0 and self.all_possible_direction(x, y, p):
                moves.append((x, y))

    return moves

def all_possible_direction(self, x: int, y: int, player: int) -> bool:
    """
    This function check the 8 possibles rays required to place a piece on the board
    :param x: the position to check in x
    :param y: the position to check in y
    :param player: the value for player pieces
    :return: if player can put a piece at the given x, y position
    """
    possible_move: bool = False

    for dx in [-1, 0, 1]:
        for dy in [-1, 0, 1]:

            if dx != 0 or dy != 0:
                possible_move = possible_move or self.possible_direction(x, y, dx, dy, player)

            if possible_move:
                return possible_move

    return possible_move

def possible_direction(self, x: int, y: int, dx: int, dy: int, player: int) -> bool:
    """
    This function check if a move is possible in a specific direction
    :param x: the x starting point of the search
    :param y: the y starting point of the search
    :param dx: the x direction
    :param dy: the y direction
    :param player: the value for player pieces
    :return: Bool: True is it is possible to play, else False
    """
```

```
    px: int = x + dx
    py: int = y + dy

    opponent: int = 1 if player == 2 else 2
    opponent_meet: bool = False  # if there were opponent pieces in the ray

    while 0 <= px < 8 and 0 <= py < 8:
        board_value = self.board[py][px]

        # if there is an empty case
        if board_value == 0:
            return False

        # if there is an opponent piece
        elif board_value == opponent:
            opponent_meet = True

        # if there is a player piece
        elif board_value == player:
            if opponent_meet:
                return True
            else:
                return False

        px += dx
        py += dy

    return False

@property
def get_board(self) -> list[list[int]]:
    return deepcopy(self.board)

@property
def get_player(self) -> int:
    return self.who_plays

# all actions on the board
def place_piece(self, x: int, y: int) -> None:
    """
```

```
        This function places a piece on the board
        :param x: the x pos of the move
        :param y: the y pos of the move
        :return: None
        """
        # check if the move is possible
        if (x, y) in self.get_possible_moves():
            # place the piece
            self.board[y][x] = self.who_plays

            self.game_history.append(self.get_board, (x, y), self.get_score, self.who_plays)

            # update the board
            self.update_board(x, y)

            # switch player
            self.switch_player()

            statu = self.game_statu
            if statu == SCORE_CANT_WHITE:
                self.who_plays = COLOR_BLACK
            elif statu == SCORE_CANT_BLACK:
                self.who_plays = COLOR_WHITE

        else:
            self.switch_player()

    @property
    def game_statu(self) -> int:
        """
        This function check if the game is finished, return PIECE_NONE if the game is not finished,
        or return the value of the piece of the winner
        :return: the type of score
        """
        board_as_list: [int] = [self.board[y][x] for y in range(8) for x in range(8)]

        # if both can't
        if len(self.get_possible_moves(COLOR_BLACK)) == 0 and len(self.get_possible_moves(COLOR_WHITE)) == 0:
            return self.get_winner
```

```
    # if black can't play
    elif len(self.get_possible_moves(COLOR_BLACK)) == 0:
        return SCORE_CANT_BLACK

    # if white can't play
    elif len(self.get_possible_moves(COLOR_WHITE)) == 0:
        return SCORE_CANT_WHITE

    if 0 in board_as_list:
        return SCORE_NOT_FINISH

    return self.get_winner

@property
def get_winner(self) -> int:
    """
    This function return the winner of the game
    :return: SCORE_WIN_W: white wins, SCORE_WIN_B: black wins, SCORE_TIE: tie
    """
    score_black, score_white = self.get_score

    if score_white > score_black or score_black <= 1:
        return SCORE_WIN_W

    elif score_white < score_black or score_white <= 1:
        return SCORE_WIN_B

    else:
        return SCORE_TIE

@property
def get_score(self) -> tuple:
    board_as_list: [int] = [self.board[y][x] for y in range(8) for x in range(8)]
    return board_as_list.count(COLOR_BLACK), board_as_list.count(COLOR_WHITE)

# all internal functions of the board
def switch_player(self) -> None:
    self.who_plays = 1 if self.who_plays == 2 else 2

def update_board(self, x: int, y: int) -> None:
```

```
        for dx in [-1, 0, 1]:
            for dy in [-1, 0, 1]:

                if dx != 0 or dy != 0:
                    self.update_board_direction(x, y, dx, dy, self.who_plays)

    def update_board_direction(self, x: int, y: int, dx: int, dy: int, player: int) -> None:
        px: int = x + dx
        py: int = y + dy

        opponent: int = 1 if player == 2 else 2

        updates: [tuple] = []

        while 0 <= px < 8 and 0 <= py < 8:
            if self.board[py][px] == opponent:
                updates.append((px, py))

            elif self.board[py][px] == player:

                for pos in updates:
                    self.board[pos[1]][pos[0]] = player

                return

            else:
                return

            px += dx
            py += dy


class GameHistory:
    """
    This class creates a game history for othello
    """

    def __init__(self) -> None:

        self.board_history: [[[int]]] = []
```

```
        self.move_history: [str] = []
        self.score_history: [tuple] = []
        self.color_history: [int] = []

        # iterator function
        self.inter_n: int = 0
        self.inter_max: int = 0

        self.move_count: int = 0  # number of move in the history

    def __iter__(self):
        self.iter_n = 0
        self.iter_max = len(self.board_history)
        return self

    def __next__(self) -> tuple:
        if self.iter_n < self.iter_max:

            return_value: tuple = (
                self.board_history[self.iter_n],
                self.move_history[self.iter_n],
                self.score_history[self.iter_n],
                self.color_history[self.iter_n]
            )
            self.iter_n += 1

            return return_value
        else:
            raise StopIteration

    def append(self, board: [[int]], move: tuple, score: tuple, color: int):
        # add element in the history
        self.board_history.append(deepcopy(board))
        self.move_history.append(deepcopy(move))
        self.score_history.append(deepcopy(score))
        self.color_history.append(color)

        self.move_count += 1

    @property
```

```
def get_move_count(self) -> int:
    return self.move_count
```

train_model.py

Entraînement des modèles

```
from copy import deepcopy
from multiprocessing import Manager
from multiprocessing import Process
import os
import random
from threading import Thread
import time

if __name__!= '__main__':
    os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

import numpy as np

from othello import Othello
from othello import COLOR_WHITE
from othello import COLOR_BLACK
from othello import SCORE_NOT_FINISH
from othello import SCORE_CANT_WHITE
from othello import SCORE_CANT_BLACK
from tensorflow_engine import board_to_input
from tensorflow_engine import TensorflowOthelloEngine

# Game properties
N_THREAD: int = 9
SAVE_DATA: bool = True

# Model properties
GAME_DEVICE: str = "/cpu:0"
PROB_RANDOM: float = 0.20  # CHANGE
MAX_DEPTH: int = 16  # CHANGE
MAX_BRANCH: int = 6  # CHANGE
TOTAL_GAME_PLAYED: int = 25514  # CHANGE
BASE_ELO: float = 4200  # CHANGE
LOAD_PREVIOUS_MODEL: bool = True  # CHANGE
COUNT_EVALUATION: bool = False
TRAIN_RATE_STOP_EXPLORATION: float = 0.10
MODEL_PATH: str = "C:/Users/Utilisateur/Documents/TIPE/tipe/global_net2.h5"

# Training properties
TRAIN_DEVICE: str = "/gpu:0"
```

```
VERBOSE: bool = False
N_EPOCHS: int = 15
N_STEPS: int = 10000
MEM_SIZE: int = 20

# Test properties
TEST_BLOCK_EVOLUTION: bool = True   # Change
TEST_DEVICE: str = "/cpu:0"
TEST_MAX_DEPTH: int = 16   # CHANGE
TEST_MAX_BRANCH: int = 6   # CHANGE
TEST_BASE_ELO: float = BASE_ELO   # CHANGE
TEST_COUNT_EVALUATION: bool = False   # CHANGE
TEST_RATE_STOP_EXPLORATION: float = 0.10

# Dashboard information
GAME_PER_LINE: int = 5
DASHBOARD_DATA: dict = {
    "train_statu": "Not started",
    "train_result": [[0.0, 0.0], [0.0, 0.0]],
    "test_statu": "Not tested",
}
EMPTY_BLOCK: str = " " * 80
NONE_BOARD = [[None for _ in range(8)] for _ in range(8)]


def pos_board(x, y):
    r = np.zeros((8, 8), dtype=np.float32)
    r[y][x] = 1.0
    return r


def new_elo_score(elo_player: float, elo_opponent: float, score) -> float:
    k: int = 20 if elo_player < 2400 else 10
    k = 20
    s: float = score
    ea: float = 1 / (1 + 10 ** ((elo_opponent - elo_player) / 400))

    return elo_player + k * (s - ea)
```

```
def dashboard():
    os.system("cls")
    while True:
        time.sleep(0.25)
        all_board = [test_board] + [v["board"] for v in m_thread_data]
        total = N_THREAD

        line_count = 0

        result = DASHBOARD_DATA["train_result"]
        print("------------------------------------------------ [Training Status] ------------------------------------------------")
        print(f"Training status: {DASHBOARD_DATA['train_statu']}{EMPTY_BLOCK}")
        print(f"Test status: {DASHBOARD_DATA['test_statu']}{EMPTY_BLOCK}")
        print(EMPTY_BLOCK, EMPTY_BLOCK)
        print(f"Results:        Loss:    Accuracy:  {EMPTY_BLOCK}")
        print(f"Network  {result[0][0]:.4f}   {result[0][1]:.4f}{EMPTY_BLOCK}")
        print(EMPTY_BLOCK, EMPTY_BLOCK)
        print("------------------------------------------------   [Game Status]   ------------------------------------------------")

        while total >= 0:

            for line in range(8):
                for g in range(GAME_PER_LINE):
                    if total - g < 0:
                        break
                    elif total - g == 0:
                        board = all_board[total - g][line]
                        print(" ".join(["T" if x == color_test else ("M" if x == color_train else ".") for x in
board]),
                              end="    ")
                    else:
                        board = all_board[total - g][line]
                        print(" ".join(["B" if x == 2 else ("W" if x == 1 else ".") for x in board]), end="     ")
                print("\n", end="")

                line_count += 1
            print(EMPTY_BLOCK)
            line_count += 1

            total -= GAME_PER_LINE
```

```
        print(f"Session game played: {sum([v['game_played'] for v in m_thread_data[:-1]])}{EMPTY_BLOCK}")
        print(f"Total game played: {TOTAL_GAME_PLAYED}{EMPTY_BLOCK}")
        print(f"Elo score: {train_model.elo_score}{EMPTY_BLOCK}")

        print("\033[F" * (line_count + 12))


def game_thread(process_id, thread_data, load_previous_model):
    # create two models
    model = TensorflowOthelloEngine(max_depth=MAX_DEPTH,
                                    device=GAME_DEVICE,
                                    train=True,
                                    rate_stop_exploration=TRAIN_RATE_STOP_EXPLORATION,
                                    prob_random=PROB_RANDOM,
                                    max_branch=MAX_BRANCH,
                                    count_evaluation=COUNT_EVALUATION)

    if load_previous_model:
        print(f"{process_id} Loading models...")
        model.global_network.load_model(MODEL_PATH)

    while thread_data[process_id]["run"]:

        if thread_data[process_id]["new_model"]:
            try:
                model.global_network.load_model(MODEL_PATH)

            except:
                pass

            thread_data[process_id]["new_model"] = False

        game = Othello()

        x_train = []
        y_train = [[], []]

        error = 0
```

```
while game.game_statu in [SCORE_NOT_FINISH, SCORE_CANT_WHITE, SCORE_CANT_BLACK]:
    board = deepcopy(game.get_board)

    thread_data[process_id]["board"] = deepcopy(board)

    x_train.append(board_to_input(board, game.who_plays))

    if game.who_plays == COLOR_BLACK:
        x, y, r = model(board, COLOR_BLACK, return_reward=True)

        y_train.append([r])

        game.place_piece(x, y)
        new_board = game.get_board

    else:
        x, y, r = model(board, COLOR_WHITE, return_reward=True)

        y_train.append([r])

        game.place_piece(x, y)
        new_board = game.get_board

    y_train[0].append(pos_board(x, y))
    y_train[1].append(r)

    # break if there is a problem with the board
    if board == new_board:
        error += 1
        if error > 3:
            break

# load every data in the manager list
thread_data[process_id]["x_train"] += x_train
thread_data[process_id]["y_train0"] += y_train[0]
thread_data[process_id]["y_train1"] += y_train[1]

# Add the 3 possible rotation
for rot in range(1, 4):
    thread_data[process_id]["x_train"] += [np.rot90(b, k=rot) for b in x_train]
```

```
            thread_data[process_id]["y_train0"] += [np.rot90(b, k=rot) for b in y_train[0]]
            thread_data[process_id]["y_train1"] += y_train[1]

        thread_data[process_id]["game_played"] += 1


if __name__== '__main__':
    manager = Manager()
    m_thread_data = manager.list()

    for i in range(N_THREAD):
        m_thread_data.append(manager.dict({
            "run": True,
            "new_model": False,
            "x_train": [],
            "y_train0": [],
            "y_train1": [],
            "game_played": 0,
            "board": NONE_BOARD
        }))

    for i in range(N_THREAD):
        Process(target=game_thread, args=(i, m_thread_data, LOAD_PREVIOUS_MODEL)).start()

    # stat training model
    train_model = TensorflowOthelloEngine(max_depth=TEST_MAX_DEPTH,
                                          device=TEST_DEVICE,
                                          train=False,
                                          rate_stop_exploration=TEST_RATE_STOP_EXPLORATION,
                                          count_evaluation=TEST_COUNT_EVALUATION,
                                          max_branch=TEST_MAX_BRANCH,
                                          base_elo=BASE_ELO)
    test_model = TensorflowOthelloEngine(max_depth=TEST_MAX_DEPTH,
                                         device=TEST_DEVICE,
                                         train=False,
                                         rate_stop_exploration=TEST_RATE_STOP_EXPLORATION,
                                         count_evaluation=TEST_COUNT_EVALUATION,
                                         max_branch=TEST_MAX_BRANCH,
                                         base_elo=TEST_BASE_ELO)
    color_test = 1
```

```
color_train = 2
test_board = NONE_BOARD

if LOAD_PREVIOUS_MODEL:
    print(f"Loading training models...")
    train_model.global_network.load_model(MODEL_PATH)

    print(f"Loading test models...")
    test_model.global_network.load_model(MODEL_PATH)

# Start dashboard
Thread(target=dashboard).start()

# Start the train loop
for i in range(N_STEPS):
    # os.system("cls")

    while sum([v['game_played'] for v in m_thread_data]) < MEM_SIZE:
        time.sleep(0.25)

    TOTAL_GAME_PLAYED += sum([v["game_played"] for v in m_thread_data])

    # get every data in every manager list
    X_TRAIN = []
    Y_TRAIN = [[], []]

    for data in m_thread_data:
        X_TRAIN += list(data["x_train"])
        Y_TRAIN[0] += list(data["y_train0"])
        Y_TRAIN[1] += list(data["y_train1"])

        data["x_train"], data["y_train1"], data["y_train0"] = [], [], []
        data["game_played"] = 0

    Y_TRAIN[0] = np.array([np.array(b).flatten() for b in Y_TRAIN[0]])

    try:
        DASHBOARD_DATA['train_statu'] = "Training Model..."

        train_model.global_network.train(np.array(X_TRAIN),
```

```
                                    [np.array(Y_TRAIN[0]), np.array(Y_TRAIN[1])],
                                    device=TRAIN_DEVICE,
                                    batch_size=1024,
                                    epochs=N_EPOCHS,
                                    verbose=VERBOSE)

        DASHBOARD_DATA['train_statu'] = "Training finished"

        # Evaluate model
        rp = train_model.global_network.network.evaluate(np.array(X_TRAIN),
                                                         [np.array(Y_TRAIN[0]), np.array(Y_TRAIN[1])],
                                                         verbose=False)
        DASHBOARD_DATA['train_result'][0][0] = rp[0]
        DASHBOARD_DATA['train_result'][0][1] = rp[1]

        # save models
        train_model.global_network.save_model(MODEL_PATH)

        # Save data history
        train_history = open("train_history.txt", "a", encoding="utf-8")
        train_history.write(f"{TOTAL_GAME_PLAYED};{rp[0]:.4f};{rp[1]:.4f};")
        train_history.close()
        tran_history_saved = True

        # save dataset
        if SAVE_DATA:
            np.savez(f"dataset/epoch_{TOTAL_GAME_PLAYED}.npz",
                     x_train=np.array(X_TRAIN),
                     y_train0=np.array(Y_TRAIN[0]),
                     y_train1=np.array(Y_TRAIN[1]),
                     allow_pickle=True)

    except Exception as e:
        tran_history_saved = False
        """tran_history_saved = False

        for t in m_thread_data:
            t["run"] = False

        train_statu = f"Train error: {e}"
```

```
    raise Exception(f"{e}")"""

    pass

    # Save the model

# Check if the model has a better score than previous version
DASHBOARD_DATA['test_statu'] = "Testing..."

test_game = Othello()
color_test = random.choice([COLOR_BLACK, COLOR_WHITE])
color_train = COLOR_BLACK if color_test == COLOR_WHITE else COLOR_WHITE

while test_game.game_statu in [SCORE_NOT_FINISH, SCORE_CANT_WHITE, SCORE_CANT_BLACK]:
    test_board = deepcopy(test_game.get_board)
    who_plays = test_game.who_plays

    if who_plays == color_test:
        pos_x, pos_y = test_model(test_board, color_test)
        test_game.place_piece(pos_x, pos_y)
    else:
        pos_x, pos_y = train_model(test_board, color_train)
        test_game.place_piece(pos_x, pos_y)

DASHBOARD_DATA['test_statu'] = "Test finished"

# Calculate new Elo rating
float_score = test_game.get_score

if color_train == COLOR_BLACK:
    float_score = float_score[0] / (float_score[0] + float_score[1])
else:
    float_score = float_score[1] / (float_score[0] + float_score[1])

# if the trained model won load the model in every models
if not TEST_BLOCK_EVOLUTION or float_score > 0.50:
    DASHBOARD_DATA['test_statu'] = "Trained model wins"
    new_network = train_model.global_network.network.get_weights()

    # load the model in every thread
```

```
        for data in m_thread_data:
            data["new_model"] = True

        # load in the test model
        test_model.global_network.network.set_weights(new_network)

        # update Elo rating
        elo_test, elo_train = test_model.get_elo, train_model.get_elo
        new_elo_train = round(new_elo_score(elo_train, elo_test, 1), 1)
        train_model.elo_score = new_elo_train

        # update test model Elo rating
        test_model.elo_score = train_model.get_elo

    else:
        DASHBOARD_DATA['test_statu'] = "Trained model loses"

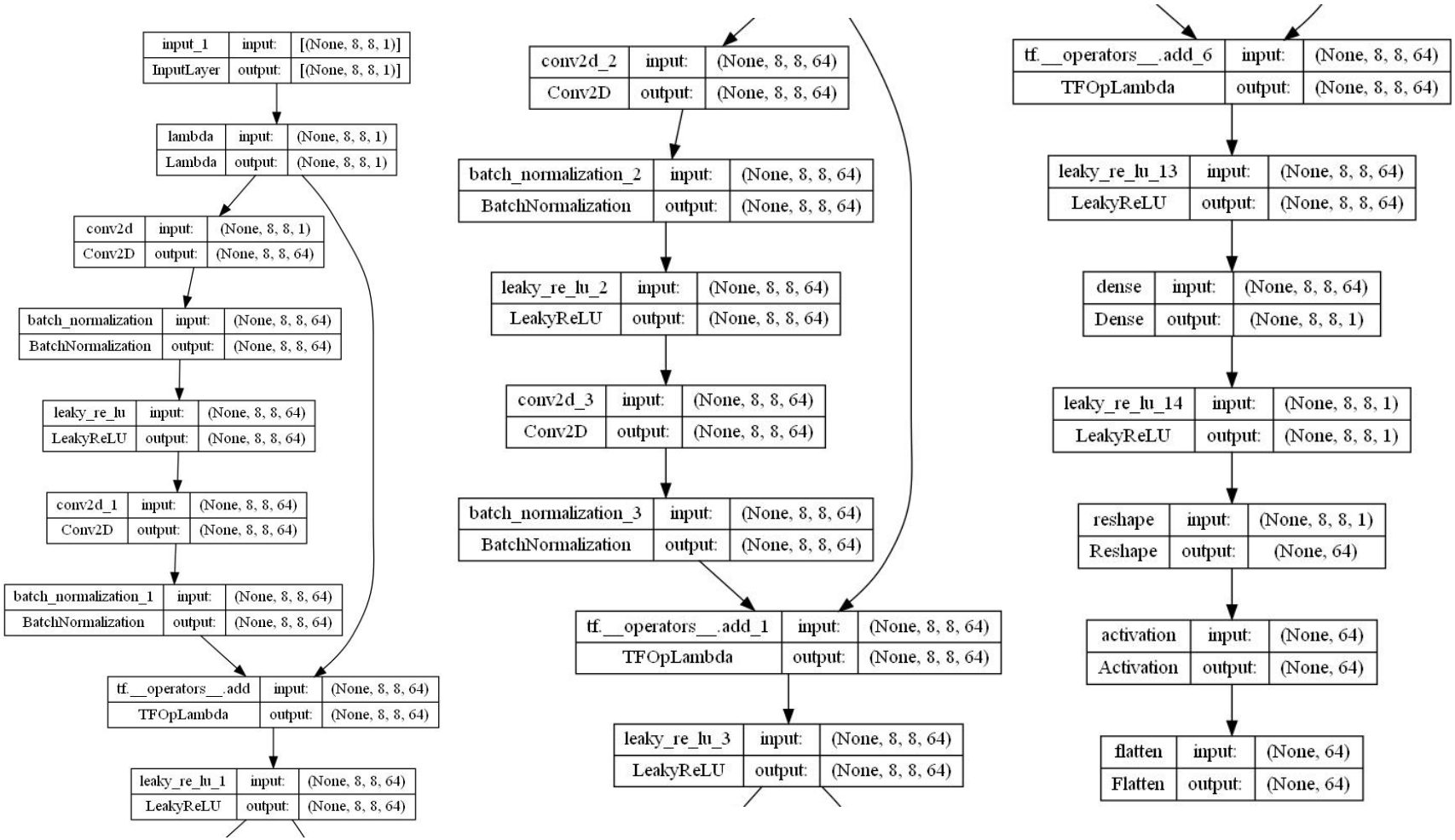
    # Save score information
    if tran_history_saved:
        train_history = open("train_history.txt", "a", encoding="utf-8")
        train_history.write(f"{float_score};{train_model.get_elo}\n")
        train_history.close()
```

Détails modèle 1

Réseaux Policy et Value

Structure détaillée

Policy network | Modèle 1



Paramètres apprentissage

Modèle 1

0 → 2529
PROB_RANDOM: float = 0.75
MAX_DEPTH: int = 10

2529 → 10’235
PROB_RANDOM: float = 0.5
MAX_DEPTH: int = 16

10’235 → 11’284 (PROBLEME)
PROB_RANDOM: float = 0.7
MAX_DEPTH: int = 12

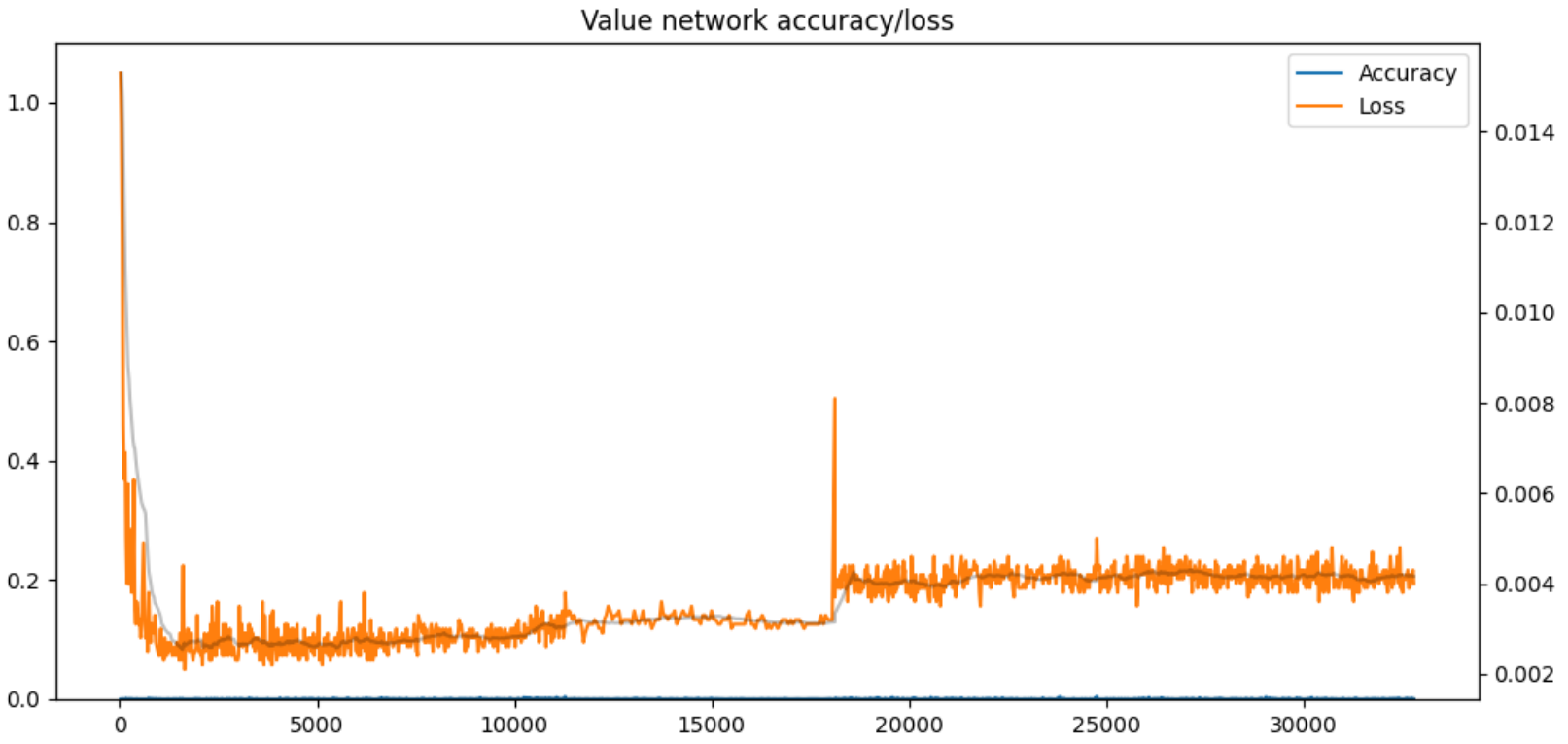
10’235 → 18’128
PROB_RANDOM: float = 0.5
MAX_DEPTH: int = 16
COUNT_EVALUATION: bool = True

18’128 → 32’811
PROB_RANDOM: float = 0.33
MAX_DEPTH: int = 16

Première victoire face à CPPEngine

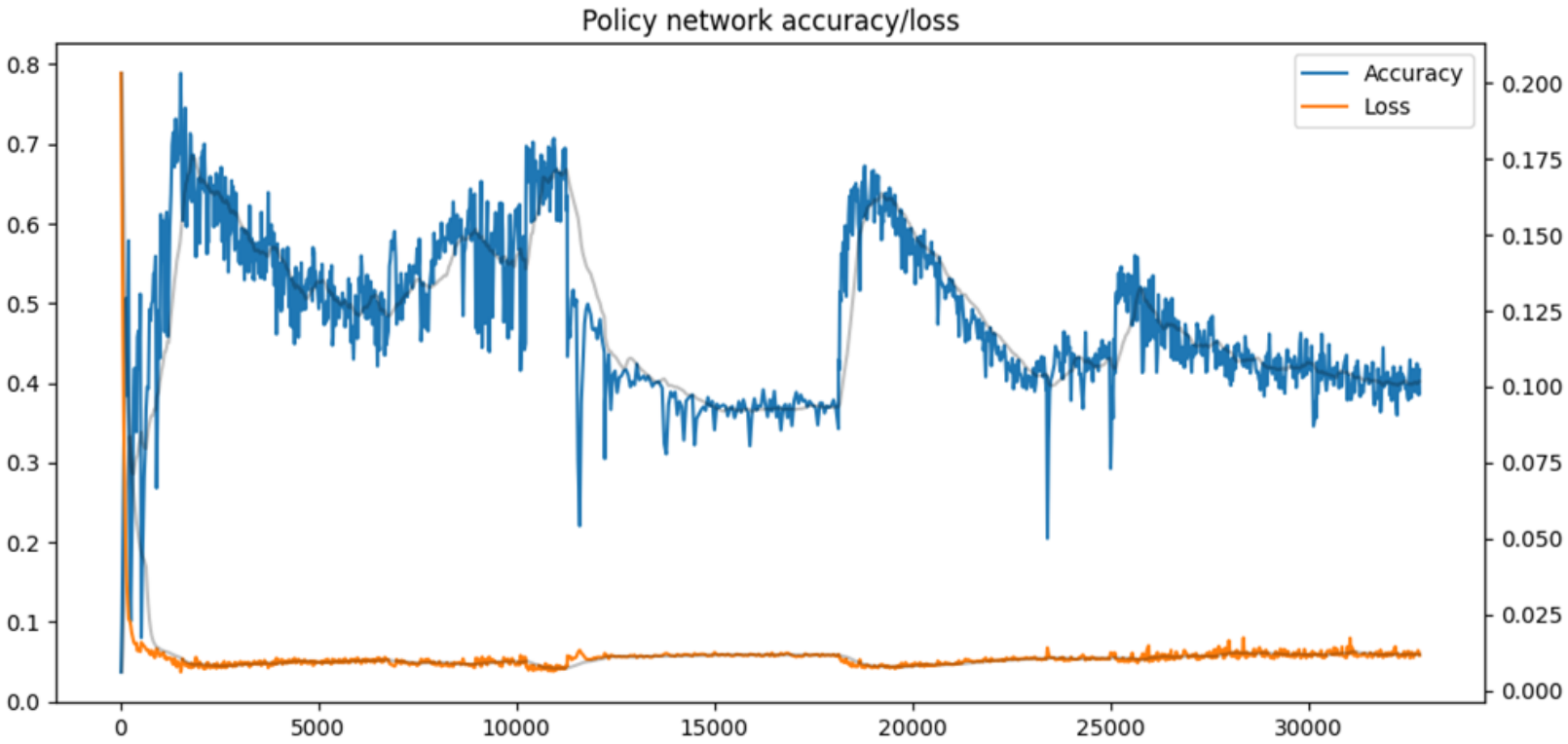
Apprentissage value network

Modèle 1



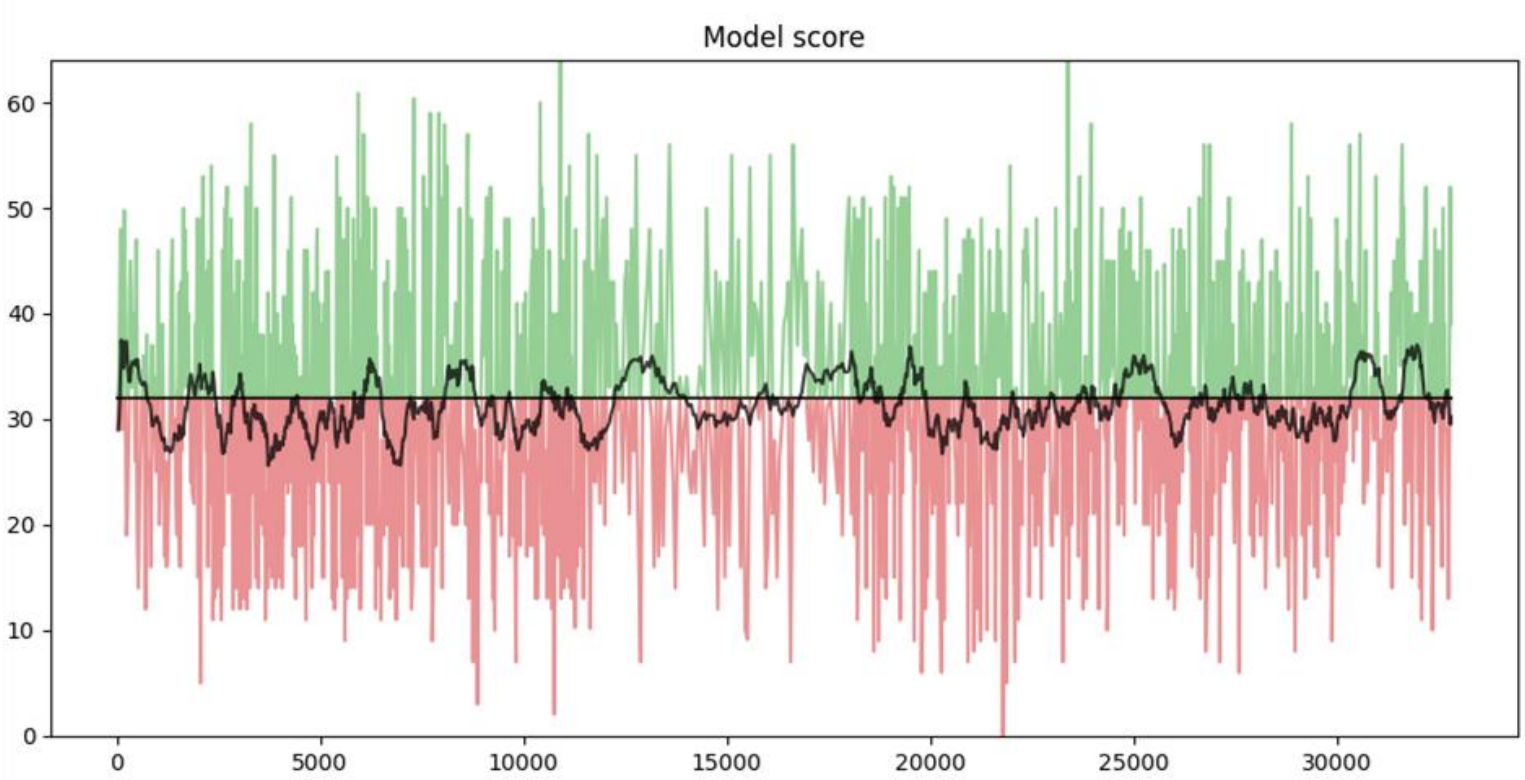
Apprentissage policy network

Modèle 1



Évolution du score

Modèle 1

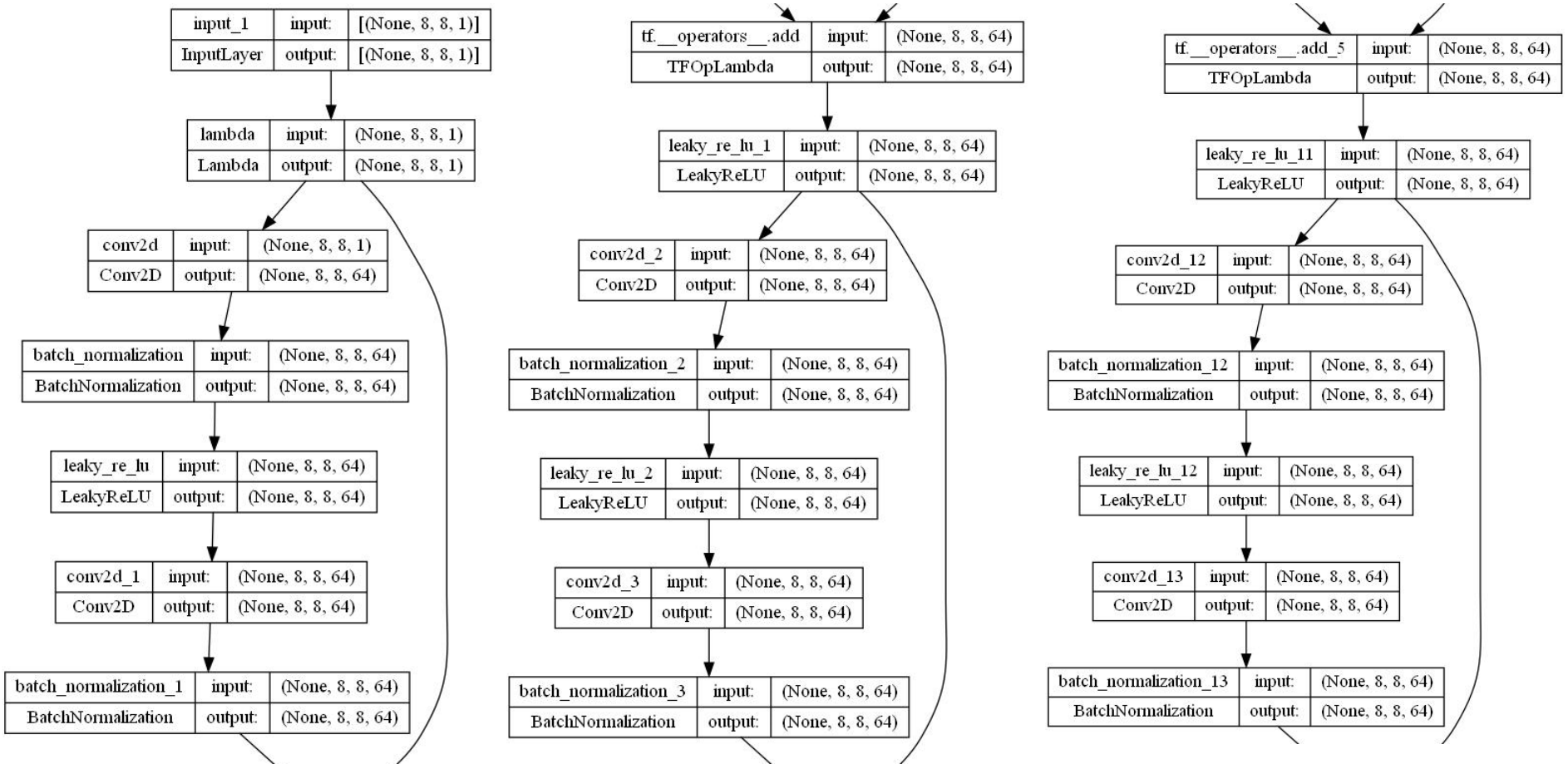


Détails modèle 2

Réseau global

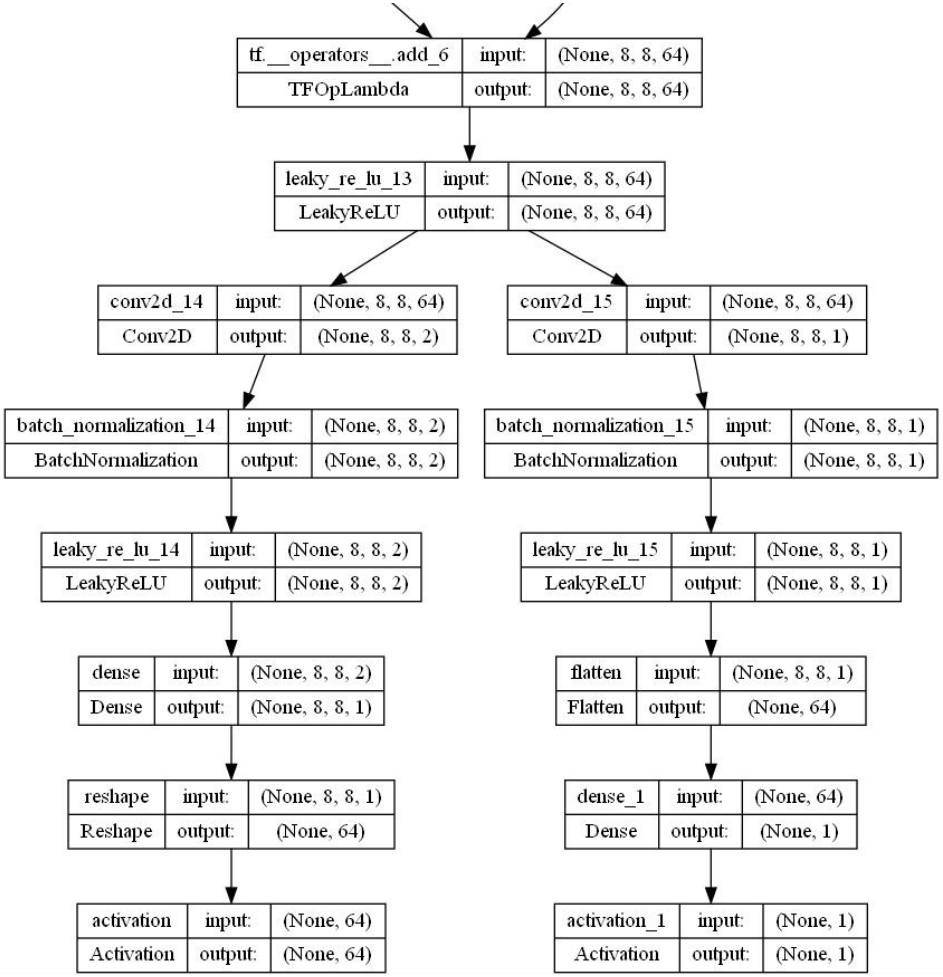
Structure détaillée

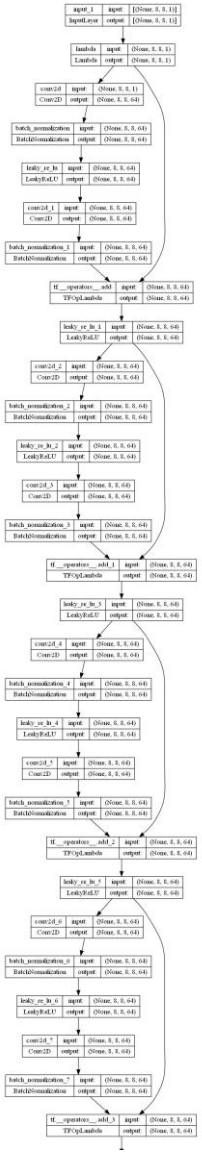
Modèle 2

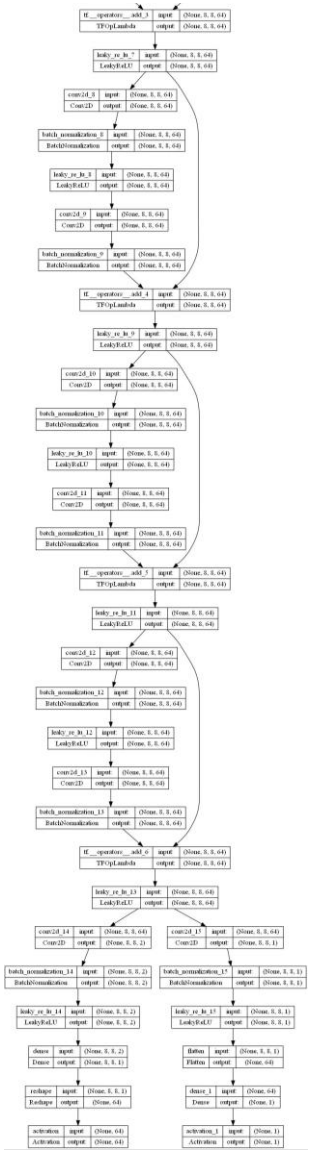


Structure détaillée

Modèle 1







Paramètres apprentissage

Modèle 2

0 → 4000 (Elo: 1150)

PROB_RANDOM: float = 0.75
MAX_DEPTH: int = 6
MAX_BRANCH: int = 6
COUNT_EVALUATION: bool = True
TRAIN_RATE_STOP_EXPLORATION: float = 0.20
N_EPOCHS: int = 25
MEM_SIZE: int = 25

4000 → 8000 (Elo: 1830)

PROB_RANDOM: float = 0.5
MAX_DEPTH: int = 10
MAX_BRANCH: int = 6
COUNT_EVALUATION: bool = True
TRAIN_RATE_STOP_EXPLORATION: float = 0.15

Première victoire face à CPPEngine

8000 → 20'000 (Elo: 3340)

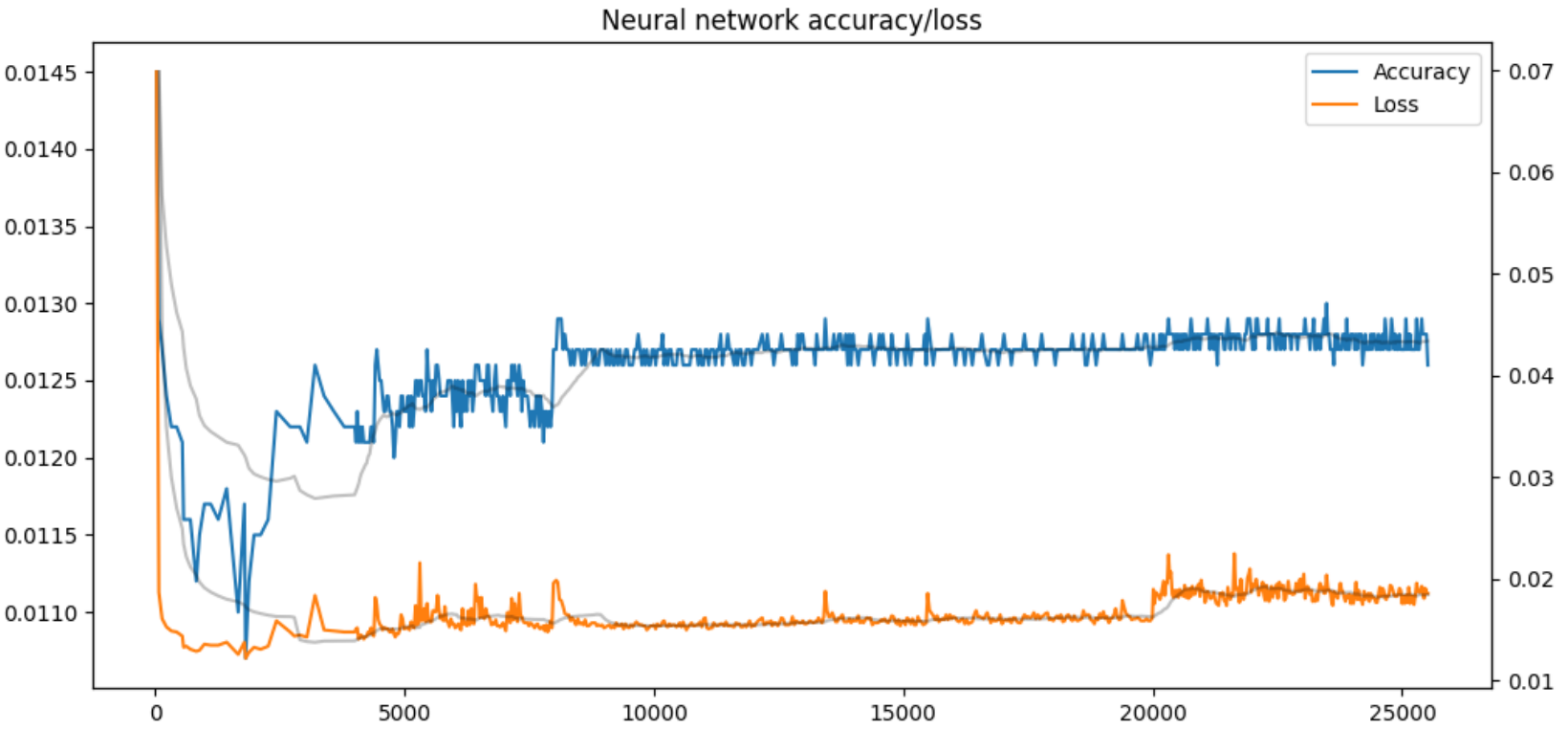
PROB_RANDOM: float = 0.33
MAX_DEPTH: int = 12
MAX_BRANCH: int = 6
COUNT_EVALUATION: bool = False
TRAIN_RATE_STOP_EXPLORATION: float = 0.10

20'000 → 26’000 (Elo: 4420)

PROB_RANDOM: float = 0.20
MAX_DEPTH: int = 16
MAX_BRANCH: int =
COUNT_EVALUATION: bool = False
TRAIN_RATE_STOP_EXPLORATION: float = 0.10
N_EPOCHS: int = 15
MEM_SIZE: int = 20

Apprentissage global network

Modèle 2



Évolution du score

Modèle 2

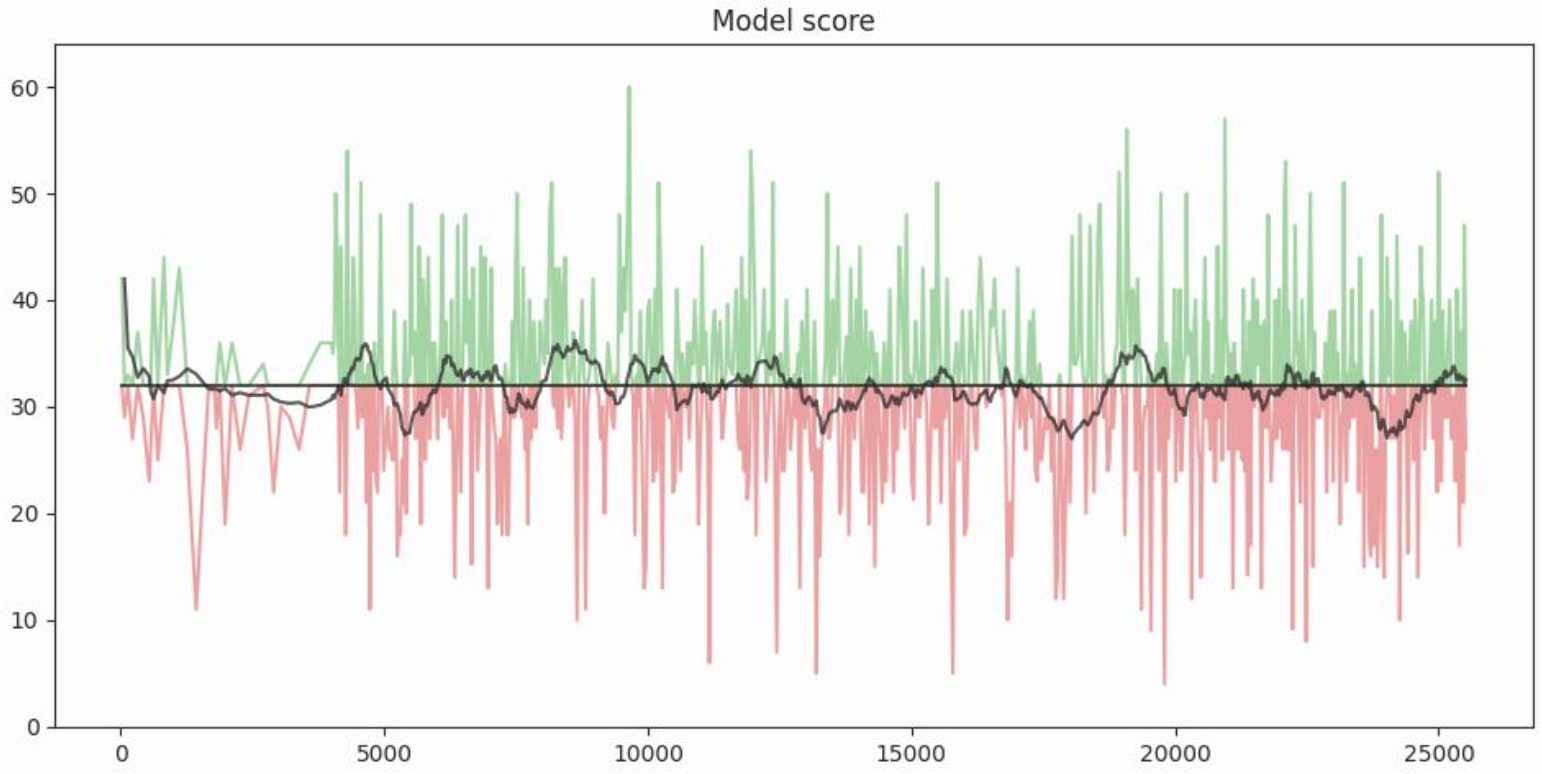


Illustration apprentissage

Modèle 2

```
------------------------------------ [Training Status] ------------------------------------
Training status: Training finished
Test status: Testing...

Results:        Loss:    Accuracy:
Network         0.0188   0.0127

------------------------------------   [Game Status]   ------------------------------------
B . . . . B W .    . B B . B . . .    . . . . . . . W    B . . . . . . .    . . B B B . . .
B . . . . B . .    . W W W B . . .    . . B . . B B B    . B . W . . . .    B B B B B B . .
B W . W W B B .    . . W W B W . .    . B B . . B . .    . W B . W . . .    . . . W B . B .
B W W W W B . .    . . W W B W . .    B W B B W B . B    . W B B B B . .    . . . B W B . .
B . W B B W . W    . . W B W W B .    W W B W B W B .    . . W W B B B W    . . B W W W . .
B W B B B W W W    . . . W B W . .    W W W B W B W B    . . . B B B W W    . B W . B W W .
W B . B B W . W    . . W B B B . .    W W . W W W W W    . . B . B B B W    B W . . B W W .
W B B B . . . .    . W . B B B B .    . . . . W W W W    . . . . B B B B    W . . . B W W .
                                                                              . . . . . .
. . W . . B W .    . B . . . . . .    . B B W . . W .    W B . . . . . W    . . T . M . . .
. . . W . W . W    . . B B . B B B    . W B W . W . .    W B B . . . . W    . T T T T T T T
W . B . W B W B    . . . B B W B B    . . W B W . . .    W B W B . . B W    M . M . T . T M
B W B W B W . .    . W W B W B . B    . . B W B B . .    . B B W B B W .    . M M T T T M M
. . W B B B B B    . W . B B W . .    . . B B W W . .    . . . B B W W W    . . M M T T T M
. . . B B B B B    W . . B B B W B    . . B W W B B B    . . . B B B B W    . . . . . T . M
. W B W W . . B    . . . B . . . W    . . B W W B . .    . . . B . B W W    . . . . . T . .
. B . W . . . .    . . . . . . . .    . . . . W . B .    . . . . B B B W    . . . . . T . .

Session game played: 15
Total game played: 25534
Elo score: 4200
```

Illustration évaluation

Modèle 2

```
TensorflowEngine is playing...
. . . B . . . .
. . . B . . . .
. B B B . . . .
. . . W B . . .
. . W W W B . .
. . B W . W . .
. . . . . . . .
. . . . . . . .

TensorflowEngine (W): 6   |   CPPEngine (B): 8
Probabilité of winning white: 0.7635780572891235
Probabilité of winning black: 0.4175942778587341
Policy network prediction:
 .  .  .  B  .  .  .  .

 .  .  .  B  .  .  .  .

 .  B  B  B  .  .  .  .

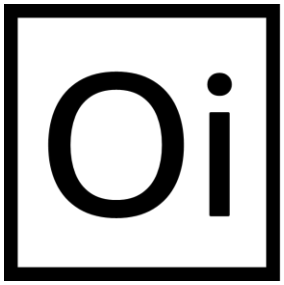
 .  .  19% W  B  .  .  .

 .  16% W  W  W  B  .  .

 .  .  B  W  17% W  .  .

 .  .  .  14% .  15% 16% .

 .  .  .  .  .  .  .  .
```



Othello AI

TIPE | Louis H. | 00000